



PABNA UNIVERSITY OF SCIENCE AND TECHNOLOGY

LAB REPORT

Course Code: ICE-3208

Course Title: Digital Image and Speech Processing Sessional

Submitted By:

Apurbo Sharma

Roll : 220604

Student of 2021-22 session

Department of ICE, PUST

Submitted To :

Dr. Md. Sarwar Hosain

Professor

Department of ICE, PUST

Dr. Md. Imran Hossain

Associate Professor

Department of ICE, PUST

Faculty of Engineering & Technology
Department of Information and Communication Engineering

Submission Date :

signature:

Index

Exp No.	Experiment Name
1.	Audio Loading and Signal Analysis.
2	Framing and Windowing (From Scratch).
3	STFT and Spectrogram Generation.
4	MFCC Extraction.
5	Pitch Detection Using Autocorrelation.
6	Speech Enhancement by Spectral Subtraction.
7	DTW-Based Isolated Word Recognition.
8	Voice Activity Detection (VAD) and Segmentation.
9	Speaker Identification using Simple Embedding.
10	Real-Time Command Recognition.
11	Apply different noise types (Gaussian, Salt & Pepper) to an image and use the following filters to remove noise: i) Mean Filter ii) Gaussian Filter Compare the effectiveness of each method. Which filter works best for which type of noise?
12	2. Perform histogram equalization on a low-contrast image. i) Compare the histogram before and after equalization. ii) How does histogram equalization affect image brightness and contrast?
13	3. Segment an object from an image using different thresholding techniques: i) Global Thresholding (Otsu's method) ii) Adaptive Thresholding iii) Compare results and discuss when adaptive thresholding is preferable over global thresholding.
14	4. Load a grayscale image and apply the following edge detection techniques: i) Sobel Operator ii) Prewitt Operator iii) Canny Edge Detector Compare the results by analyzing the sharpness and completeness of detected edges.
15	Write a MATLAB/Python program to Character Segment of an image.
16	Write a MATLAB/Python program to read 'rice.tif' image, count number of rice and display area (also specific range), major axis length, and perimeter.
17	Write a MATLAB/Python program to read an image and perform convolution with 3X3 mask.
18	Develop a MATLAB/Python program that performs a two-scale image decomposition using the wavelet transform, and generate plots to visualize the results.

Experiment-1: Audio Loading and Signal Analysis

Using Python, read a given speech .wav file and perform:

- a) Print sampling rate, number of samples, duration, max/min amplitude.
- b) Plot the waveform with proper axis labeling.
- c) Compute and plot magnitude spectrum using FFT.

Objectives:

1. To read a speech signal from a .wav audio file using Python.
2. To extract and display basic signal information such as sampling rate, duration, and amplitude range.
3. To visualize the speech waveform in the time domain.
4. To compute and analyze the frequency components of the speech signal using Fast Fourier Transform (FFT).
5. To plot the magnitude spectrum of the speech signal.

Theory:

A speech signal is a time-varying waveform that contains information in both time and frequency domains. Digital speech processing involves analyzing these properties using computational tools.

Sampling Rate:

The sampling rate represents the number of samples recorded per second (Hz). It determines the resolution of the digital signal.

Waveform:

The waveform shows amplitude variations with respect to time. It helps to observe speech characteristics such as silence, voiced, and unvoiced regions.

Fast Fourier Transform (FFT):

FFT is an efficient algorithm used to convert a time-domain signal into its frequency-domain representation. It reveals the distribution of frequency components present in the speech signal.

Magnitude Spectrum:

The magnitude spectrum represents the strength (amplitude) of each frequency component. Speech signals typically have most energy concentrated in lower frequency ranges.

Output:

Basic Information

Sampling Rate: 44100 Hz

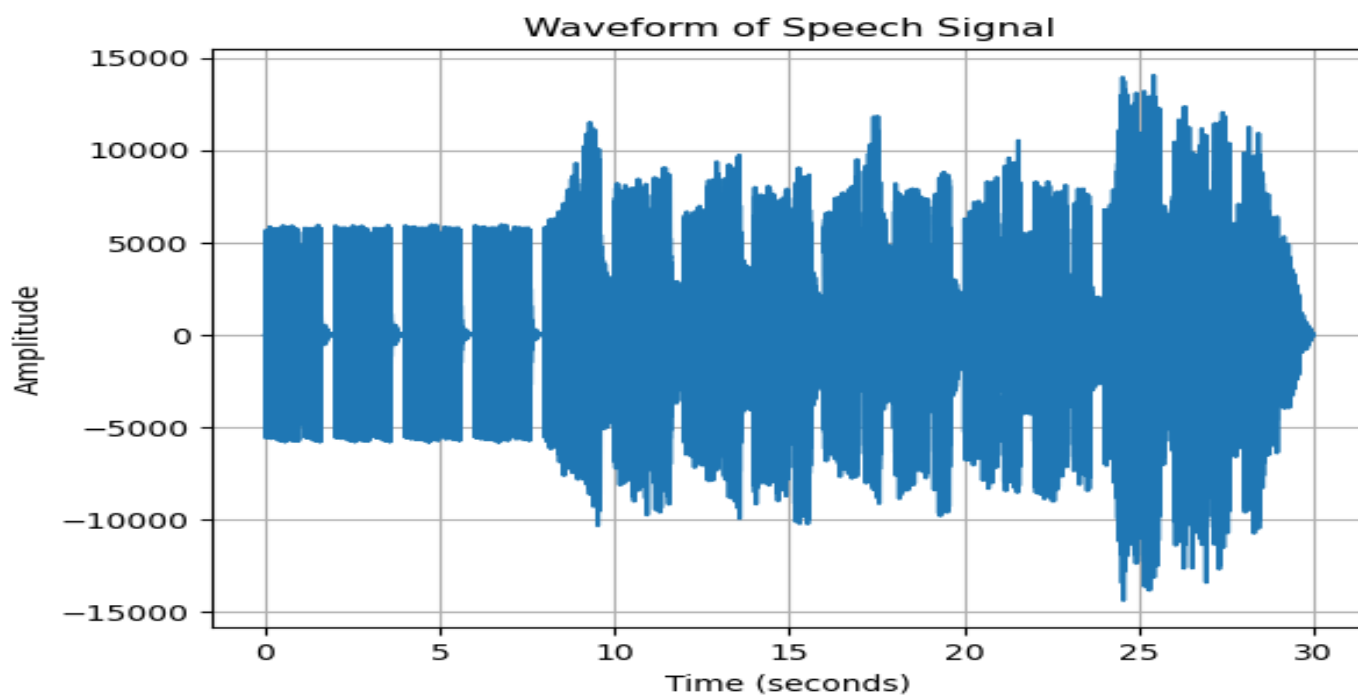
Number of Samples: 1322253

Duration: 29.983 seconds

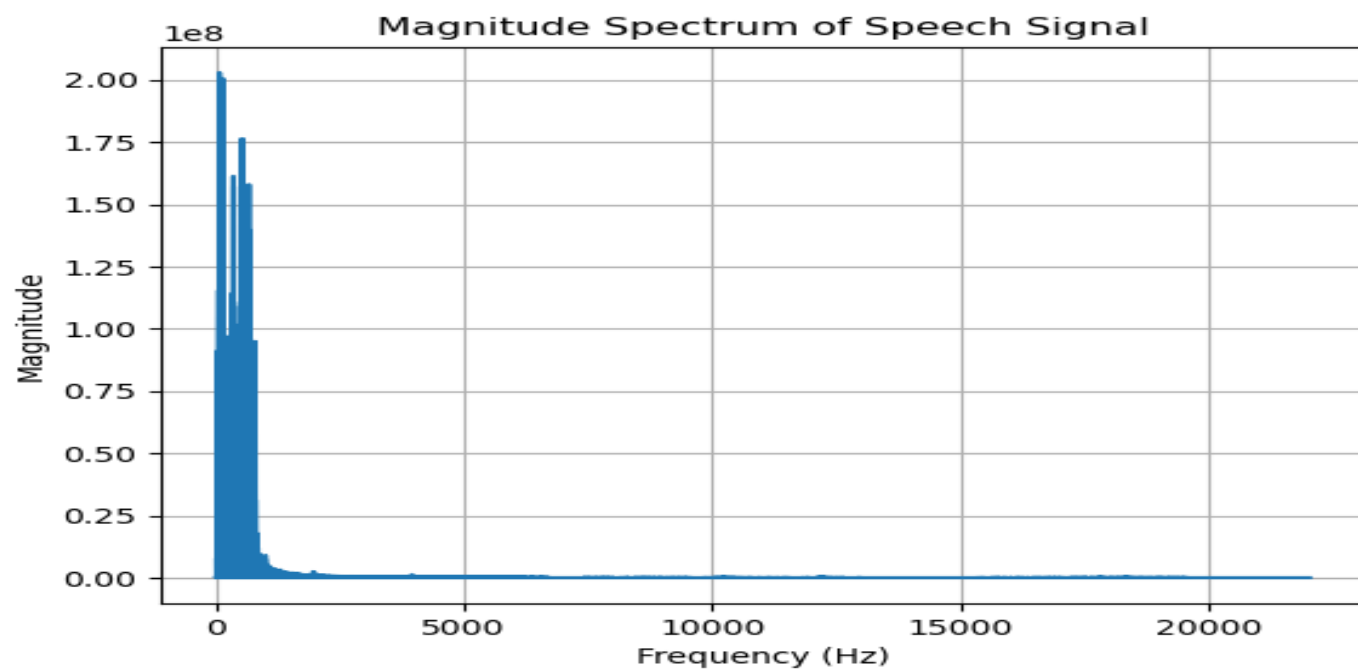
Maximum Amplitude: 14040

Minimum Amplitude: -14377

Waveform Plot:



Magnitude Spectrum:



Experiment-2: Framing and Windowing (From Scratch)

Write Python code to segment a speech signal into frames:

- a) Frame size = 25 ms, frame shift = 10 ms.
- b) Apply Hamming window to each frame.
- c) Show total number of frames and plot two sample frames (before & after window).

Objectives:

1. To read a speech signal from a .wav file.
2. To segment the speech signal into short-time frames using fixed frame size and frame shift.
3. To implement the Hamming window manually.
4. To apply the window function to each frame.
5. To compare the original frame and windowed frame visually.

Theory:

Speech signals are non-stationary in nature, meaning their statistical properties change over time. However, over short durations (20–30 ms), speech can be considered approximately stationary. Therefore, speech processing techniques analyze signals using short-time frames.

Framing:

Framing is the process of dividing a continuous speech signal into small overlapping segments called frames. Typically, a frame size of 20–30 ms with a shift of 10 ms is used. Overlapping ensures smooth transitions between frames and preserves information.

Windowing:

When a signal frame is extracted, discontinuities may occur at the frame boundaries. Windowing reduces these discontinuities by tapering the signal edges toward zero.

Hamming Window:

The Hamming window is commonly used in speech processing because it minimizes spectral leakage. It is defined as:

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right)$$

where N is the frame length and n is the sample index.

Applying a window improves frequency analysis accuracy in later processing steps such as FFT and MFCC.

Output:

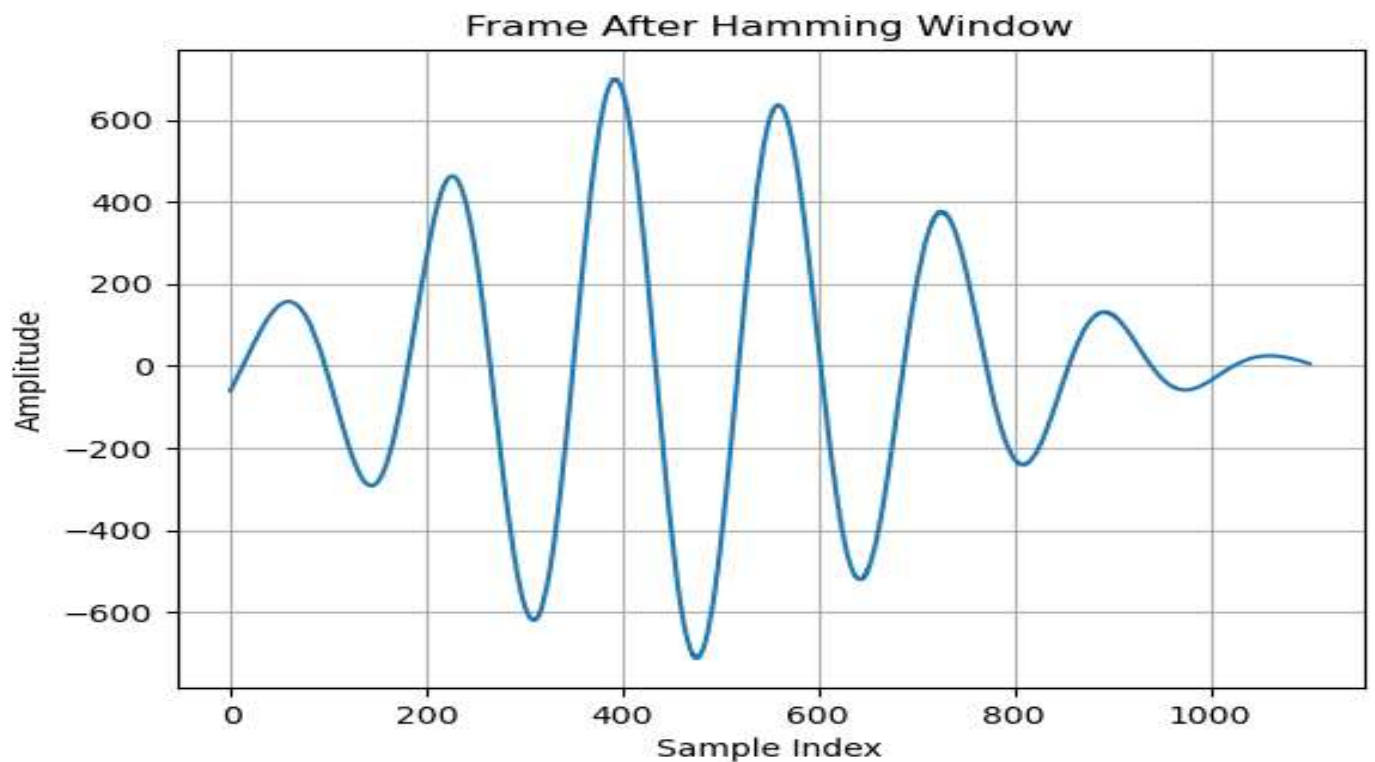
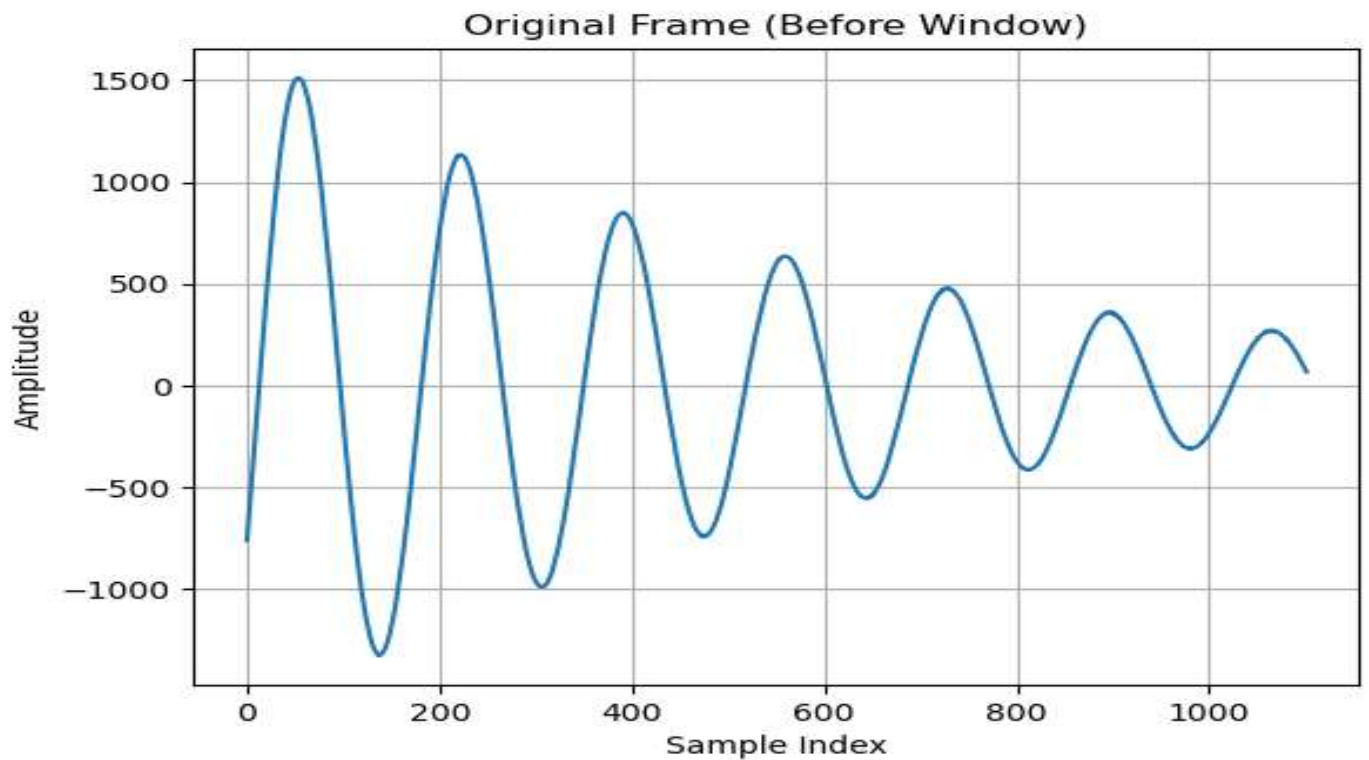
Frame Information

Frame size (samples): 1102

Frame shift (samples): 441

Total number of frames: 2996

Plots:



Experiment-3: STFT and Spectrogram Generation

Generate a spectrogram of the given speech signal:

- a) Implement STFT using frame-wise FFT (rfft).
- b) Convert magnitude to dB and plot the spectrogram.
- c) Write one practical application of spectrogram in speech processing.

Objectives:

1. To read a speech signal from a .wav file.
2. To segment the signal into short-time frames using frame size and frame shift.
3. To apply the Hamming window to each frame.
4. To compute the Short-Time Fourier Transform (STFT) using frame-wise FFT.
5. To convert the magnitude spectrum into decibel (dB) scale.
6. To generate and visualize the spectrogram of the speech signal.

Theory:

Speech signals are time-varying (non-stationary), meaning their frequency content changes over time. The Short-Time Fourier Transform (STFT) is used to analyze how frequency components evolve over time by applying Fourier Transform on short overlapping frames of the signal.

Short-Time Fourier Transform (STFT):

STFT divides a signal into small frames, applies a window function, and computes the Fourier Transform for each frame. This produces a time-frequency representation of the signal.

Mathematically:

$$STFT(n, k) = \sum_{m=0}^{N-1} x[m] \cdot w[n - m] \cdot e^{-j2\pi km/N}$$

where

- $x[m]$ = input signal
- $w[n]$ = window function
- N = frame length

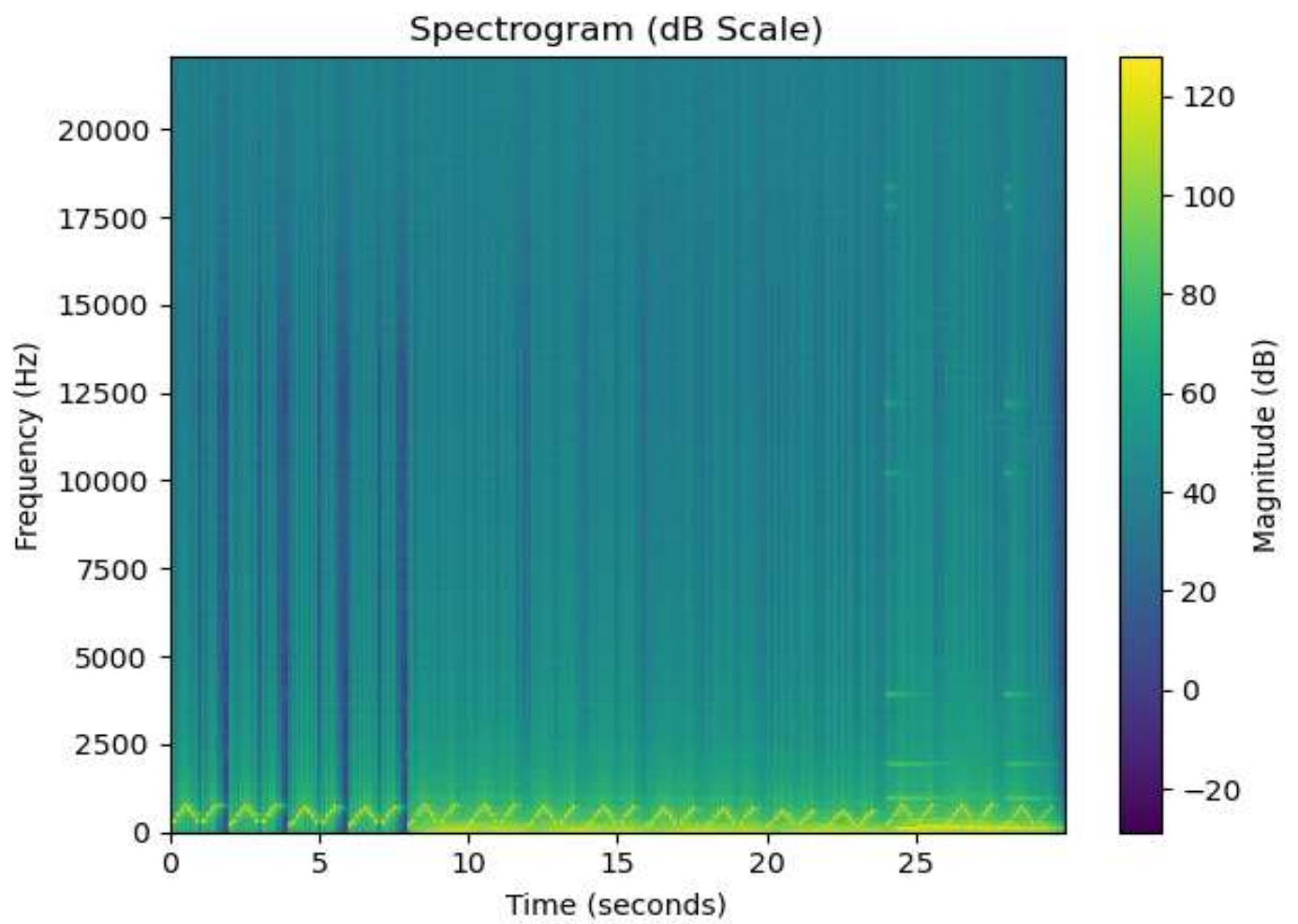
Spectrogram:

A spectrogram is the visual representation of STFT magnitude over time and frequency. The magnitude is often converted to decibels (dB) to better represent human perception of sound intensity.

Lower frequencies typically contain most speech energy, while higher frequencies provide details such as consonants.

Output:

Spectrogram:



Experiment-4: MFCC Extraction

Implement MFCC extraction:

- a) Implement the steps: pre-emphasis → framing → FFT → Mel filterbank → log → DCT.
- b) Extract 13 MFCCs per frame and plot MFCC heatmap.
- c) Mention why MFCC is effective for speech-related tasks (2–3 lines).

Objectives:

1. To read a speech signal from a .wav file and preprocess it.
2. To apply pre-emphasis to enhance high-frequency components.
3. To segment the signal into short-time frames and apply a Hamming window.
4. To compute the FFT and power spectrum of each frame.
5. To construct and apply Mel filter banks.
6. To compute MFCC features using Discrete Cosine Transform (DCT).
7. To visualize MFCC coefficients using a heatmap.

Theory:

Mel-Frequency Cepstral Coefficients (MFCCs) are widely used features in speech and audio processing, particularly in speech recognition systems. They represent the short-term power spectrum of sound based on human auditory perception.

Pre-emphasis:

Pre-emphasis amplifies high-frequency components to balance the speech spectrum. It is defined as:

$$y[n] = x[n] - \alpha x[n-1]$$

where α is typically 0.95–0.97.

Framing and Windowing:

The speech signal is divided into short frames (20–30 ms) and windowed to reduce discontinuities at boundaries.

FFT and Power Spectrum:

FFT converts the time-domain signal into frequency domain, and power spectrum represents signal energy at different frequencies.

Mel Scale:

The Mel scale approximates human auditory perception of frequency. Conversion between frequency (Hz) and Mel is:

$$Mel = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

Mel Filterbank:

Triangular filters are applied in Mel scale to capture perceptually important frequency components.

Discrete Cosine Transform (DCT):

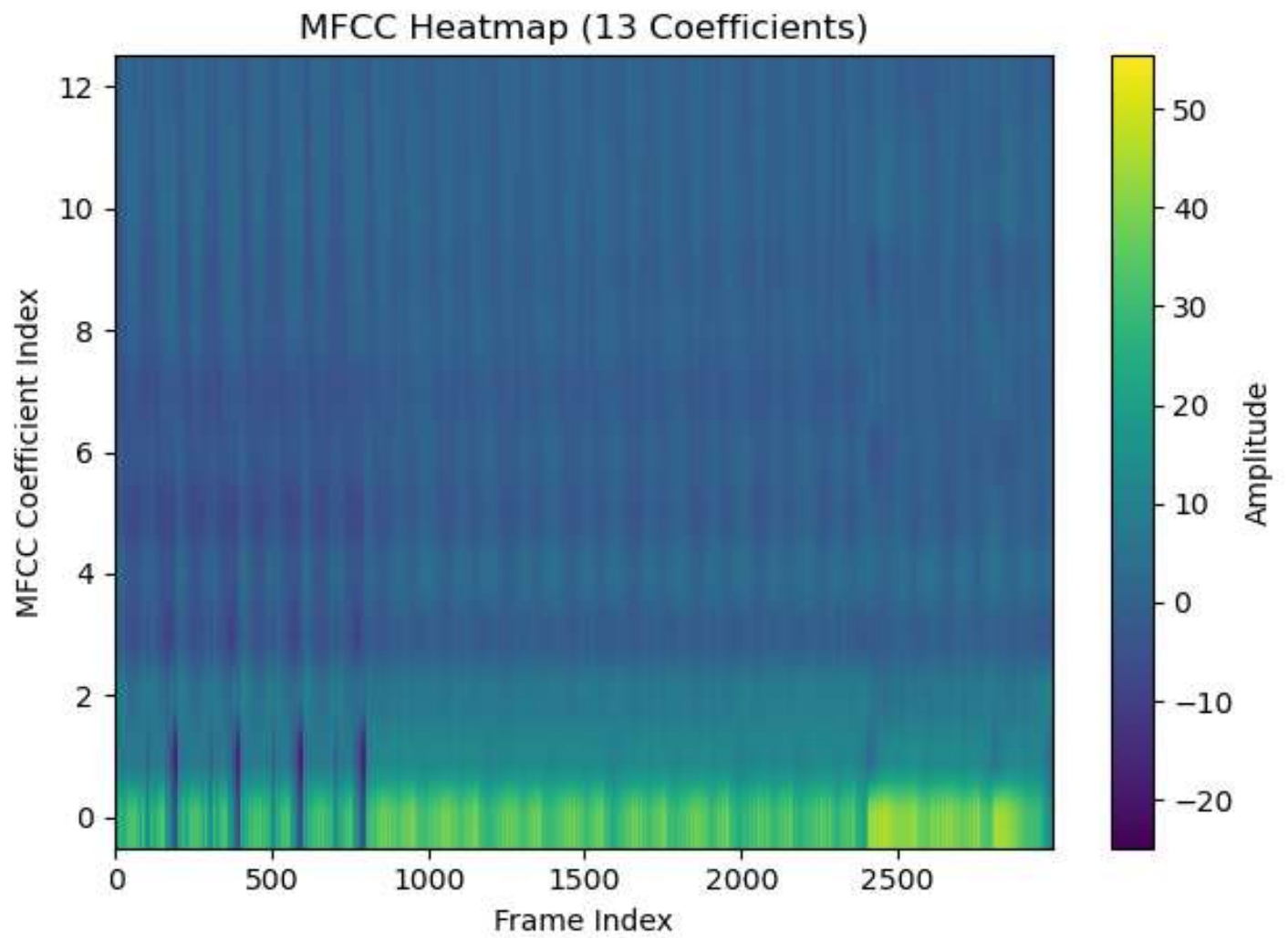
DCT decorrelates filterbank energies and produces MFCC features. Usually, the first 12–13 coefficients are used.

MFCCs are effective because they capture the spectral envelope of speech signals.

Output:

MFCC shape: (2996, 13)

MFCC Heatmap



Experiment-5: Pitch Detection Using Autocorrelation

Estimate pitch (F0) from speech:

- a) Implement autocorrelation-based pitch estimation per frame.
- b) Apply voiced/unvoiced detection using energy threshold.
- c) Plot pitch contour (F0 vs time) for voiced frames.

Objectives:

1. To read a speech signal from a .wav file and preprocess it.
2. To divide the speech signal into short-time frames.
3. To estimate pitch (fundamental frequency, F0) using the autocorrelation method for each frame.
4. To perform voiced/unvoiced detection using an energy threshold.
5. To plot the pitch contour (F0 vs time) of the speech signal.

Theory:

Pitch is the perceptual attribute of sound that corresponds to the fundamental frequency (F0) of a speech signal. It is an important feature in speech processing applications such as speech recognition, speaker identification, and emotion detection.

Autocorrelation Method:

Autocorrelation measures the similarity between a signal and a delayed version of itself. For periodic signals like voiced speech, peaks appear at delays corresponding to the pitch period.

Mathematically:

$$R(\tau) = \sum_n x(n) \cdot x(n - \tau)$$

where

- $x(n)$ = speech signal
- τ = time lag

The pitch frequency is calculated

$$F_0 = \frac{f_s}{\text{lag}}$$

where f_s is the sampling frequency.

Pitch Range:

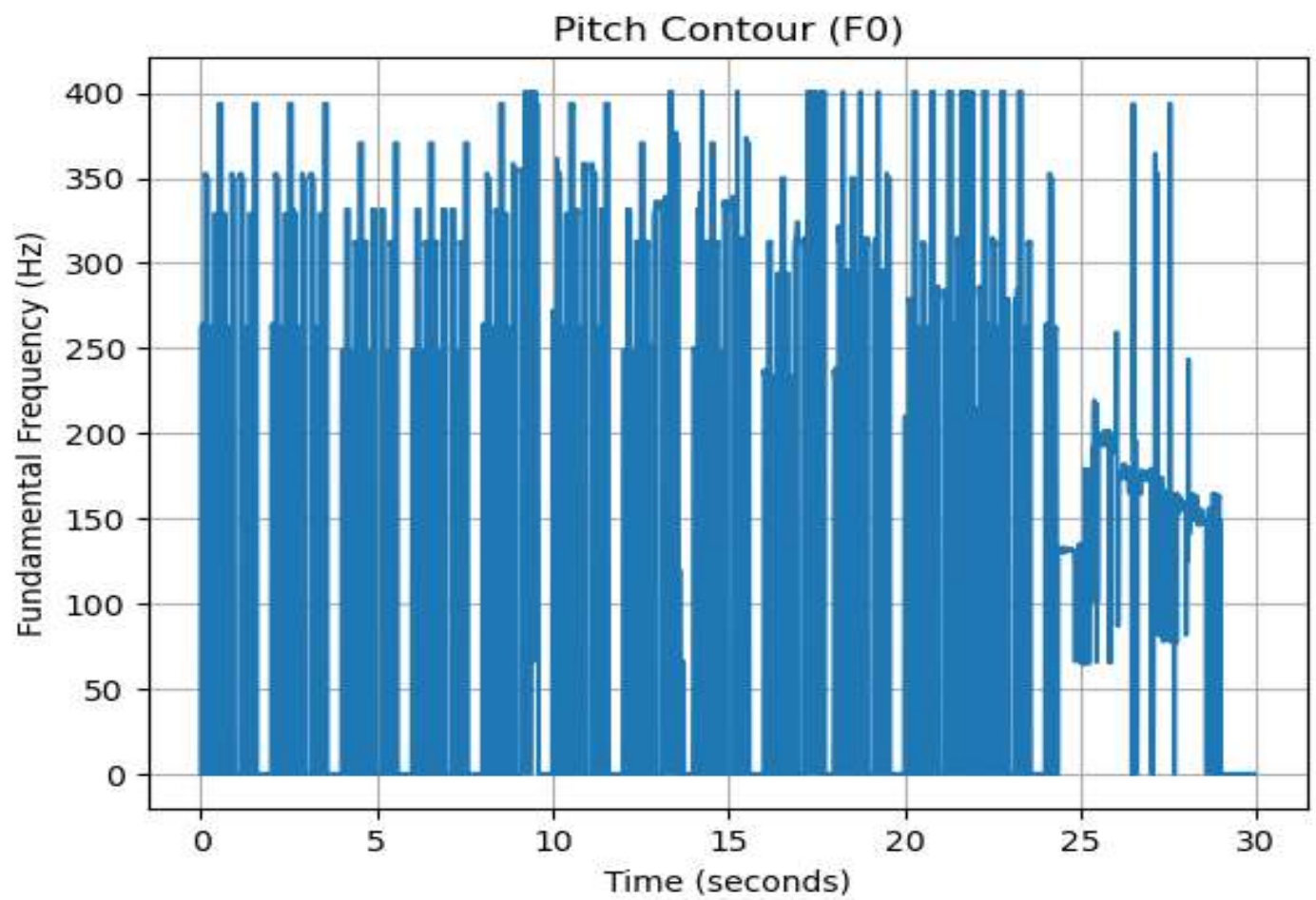
Human speech typically has pitch frequencies between 50 Hz and 400 Hz. The algorithm searches within this range to find the most likely pitch.

Voiced/Unvoiced Detection:

Voiced speech has higher energy than unvoiced speech. Energy thresholding is used to identify voiced frames. Pitch values for unvoiced frames are set to zero.

Output:

Pitch Contour



Problem No - 06 : Speech Enhancement by Spectral Subtraction. Noise reduction task:

- Add white noise to the speech at SNR = 10 dB (or given SNR).
- Implement spectral subtraction for enhancement.
- Compute SNR before and after enhancement and report improvement.

Objectives

- To add white Gaussian noise to a clean speech signal at a specified SNR level (10 dB).
- To implement the spectral subtraction algorithm for noise reduction in the frequency domain.
- To reconstruct the enhanced time-domain speech signal using the inverse FFT.
- To compute the Signal-to-Noise Ratio (SNR) before and after enhancement.
- To quantify and report the SNR improvement achieved by spectral subtraction.

Theory

Spectral subtraction is a classical speech enhancement technique that operates in the frequency domain. The core idea is to estimate the noise spectrum from a noise-only region (typically the first few frames of a signal) and subtract it from the noisy speech spectrum.

Given a clean speech signal $s(t)$ and additive noise $n(t)$, the noisy observation is:

$$x(t) = s(t) + n(t) \quad (1)$$

To add noise at a desired SNR (in dB), the noise is scaled as:

$$k = \sqrt{\frac{P_s}{10^{\text{SNR}/10} \cdot P_n}} \quad (2)$$

where P_s and P_n are the power of the clean signal and the unit-variance noise, respectively.

The Short-Time Fourier Transform (STFT) is applied frame-by-frame using a Hamming window. For each frame, the magnitude spectrum is computed:

$$|X(\omega)| = |S(\omega) + N(\omega)| \quad (3)$$

The noise estimate $|\hat{N}(\omega)|$ is obtained from the average magnitude of the first few (noise-only) frames. The clean magnitude spectrum is estimated by:

$$|\hat{S}(\omega)| = \max(|X(\omega)| - |\hat{N}(\omega)|, 0) \quad (4)$$

Half-wave rectification (clamping to zero) avoids negative values, which are physically meaningless. The enhanced frame is reconstructed by combining the estimated magnitude with the original phase:

$$\hat{S}(\omega) = |\hat{S}(\omega)| \cdot e^{j\angle X(\omega)} \quad (5)$$

and the time-domain signal is recovered via the inverse FFT (IFFT). The SNR is computed as:

$$\text{SNR} = 10 \log_{10} \left(\frac{P_s}{P_{\text{noise}}} \right) \text{ dB} \quad (6)$$

Source Code :

```
noise = np.random.normal(0, 1, len(signal))
signal_power = np.mean(signal**2) noise_power
= np.mean(noise**2)

k = np.sqrt(signal_power / (10**(10/10) * noise_power)) noise
= noise * k noisy = signal + noise
```

Listing 1: Adding White Noise at 10 dB SNR

```

noise_spec = np.mean(np.abs(rfft(noisy[:frame_size])))

enhanced = []
for frame in frames:
    spec = np.abs(rfft(frame))
    clean = spec - noise_spec
    clean[clean < 0] = 0
    enhanced.append(clean)

```

Listing 2: Spectral Subtraction – Frame-level Enhancement

```

noise_before = noisy - signal
signal_power = np.mean(signal**2)
noise_power_before = np.mean(noise_before**2)
snr_before = 10 * np.log10(signal_power / noise_power_before)
print("SNR Before Enhancement:", snr_before, "dB")

```

Listing 3: SNR Computation Before Enhancement

```

enhanced_signal = np.zeros(len(signal))

for i in range(0, len(signal) - frame_size, frame_shift):
    frame = noisy[i:i + frame_size]
    windowed = frame * np.hamming(frame_size)

    spectrum = np.fft.rfft(windowed)
    mag = np.abs(spectrum)
    phase = np.angle(spectrum)

    noise_est = np.mean(np.abs(np.fft.rfft(noisy[:frame_size])))
    clean_mag = mag - noise_est
    clean_mag[clean_mag < 0] = 0

    clean_spectrum = clean_mag * np.exp(1j * phase)
    clean_frame = np.fft.irfft(clean_spectrum)
    enhanced_signal[i:i + frame_size] += clean_frame

noise_after = enhanced_signal - signal
noise_power_after = np.mean(noise_after**2)
snr_after = 10 * np.log10(signal_power / noise_power_after)
print("SNR After Enhancement:", snr_after, "dB")
print("SNR Improvement:", snr_after - snr_before, "dB")

```

Listing 4: Full Enhancement Pipeline and SNR After

Output

```

SNR Before Enhancement : 8.39 dB
SNR After Enhancement   : 12.60 dB
SNR Improvement          : +3.23 dB

```

Result & Discussion:

The result confirms that spectral subtraction effectively reduces stationary additive noise. However, the method assumes stationary noise and accurate noise estimation from initial frames, which may not hold in practical scenarios. This can introduce “musical noise” artifacts—residual tonal distortions that degrade perceptual quality. More advanced methods such as Wiener filtering and deep learning-based enhancement can mitigate these limitations.

Problem No - 07 : DTW-Based Isolated Word Recognition. Build a small word recognizer:

- Prepare dataset of 4 words with 3 training and 2 testing samples each.
- Extract MFCC sequences and classify using DTW distance.
- Report accuracy and draw confusion matrix.

Objectives

- To build a small isolated word recognizer using a dataset of four spoken words.
- To extract Mel-Frequency Cepstral Coefficient (MFCC) sequences as acoustic features.
- To implement Dynamic Time Warping (DTW) as a distance metric for template matching.
- To classify test utterances by finding the nearest training template using DTW.
- To evaluate recognizer performance by computing accuracy and drawing a confusion matrix.

Theory

Isolated word recognition matches an unknown spoken word against a set of stored templates. MFCC features capture the short-term spectral envelope of speech and are robust to speaker variability. For each utterance, a sequence of 13-dimensional MFCC vectors is extracted frame-by-frame.

Because different utterances of the same word may have different durations, a simple Euclidean distance between fixed-length vectors is insufficient. **Dynamic Time Warping (DTW)** solves this by finding the optimal non-linear alignment between two sequences $\mathbf{X} = (x_1, \dots, x_M)$ and $\mathbf{Y} = (y_1, \dots, y_N)$.

A local distance matrix is first computed:

$$D(i, j) = \|x_i - y_j\|_2 \quad (7)$$

The cumulative cost matrix C is then filled using dynamic programming with the recurrence:

$$C(i, j) = D(i, j) + \min(C(i-1, j), C(i, j-1), C(i-1, j-1)) \quad (8)$$

with boundary conditions $C(0,0) = D(0,0)$, $C(i,0) = \min_{k=0}^{i-1} D(k,0)$, and $C(0,j) = \min_{k=0}^{j-1} D(0,k)$. The DTW distance is $C(M-1, N-1)$, the cost of the optimal warping path.

Classification uses the 1-nearest-neighbour rule: a test sample is assigned the label of the training template that yields the minimum DTW distance.

Source Code

```
def extract_mfcc(file):  
    y, sr = librosa.load(file, sr=16000)  
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)  
    return mfcc.T
```

Listing 5: MFCC Extraction

```
def dtw_distance(x, y):  
    D = cdist(x, y)  
    cost = np.zeros(D.shape)  
    cost[0, 0] = D[0, 0]
```

```

for i in range(1, D.shape[0]): cost[i, 0] = D[i, 0]
    + cost[i-1, 0]
for j in range(1, D.shape[1]): cost[0, j] = D[0, j]
    + cost[0, j-1]
for i in range(1, D.shape[0]):
    for j in range(1, D.shape[1]): cost[i, j] = D[i,
        cost[i-1, j], cost[i, j-1], cost[i-1, j-1]
    )
return cost[-1, -1]
j] + min(

```

Listing 6: DTW Distance Computation

```

DATASET_PATH = "mini_speech_commands/mini_speech_commands"
WORDS = ["left", "right", "stop", "go"]
FILES_PER_WORD = 40
TRAIN_PER_WORD = 15
TEST_PER_WORD = 10

train_data, train_labels = [], []
test_data, test_labels = [], []

for word in WORDS:
    folder = os.path.join(DATASET_PATH, word)
    files = os.listdir(folder)[:FILES_PER_WORD]
    np.random.shuffle(files)

    for i, file in enumerate(files):
        mfcc = extract_mfcc(os.path.join(folder, file))
        if i < TRAIN_PER_WORD:
            train_data.append(mfcc)
            train_labels.append(word)
        elif i < TRAIN_PER_WORD + TEST_PER_WORD:
            test_data.append(mfcc)
            test_labels.append(word)

```

Listing 7: Dataset Preparation

```

predictions = []
for test_sample in test_data:
    distances = [dtw_distance(test_sample, t) for t in train_data]
    predictions.append(train_labels[np.argmin(distances)])

accuracy = accuracy_score(test_labels, predictions)
print("Final Accuracy:", accuracy)

cm = confusion_matrix(test_labels, predictions, labels=WORDS)
sns.heatmap(cm, annot=True, fmt="d", xticklabels=WORDS, yticklabels=WORDS)
plt.xlabel("Predicted Label"); plt.ylabel("True Label")
plt.title("Confusion Matrix (DTW Word Recognition)")
plt.show()

```

Listing 8: 1-NN Classification and Evaluation

Output

Total Train Samples : 70
Total Test Samples : 30
Final Accuracy : 0.453

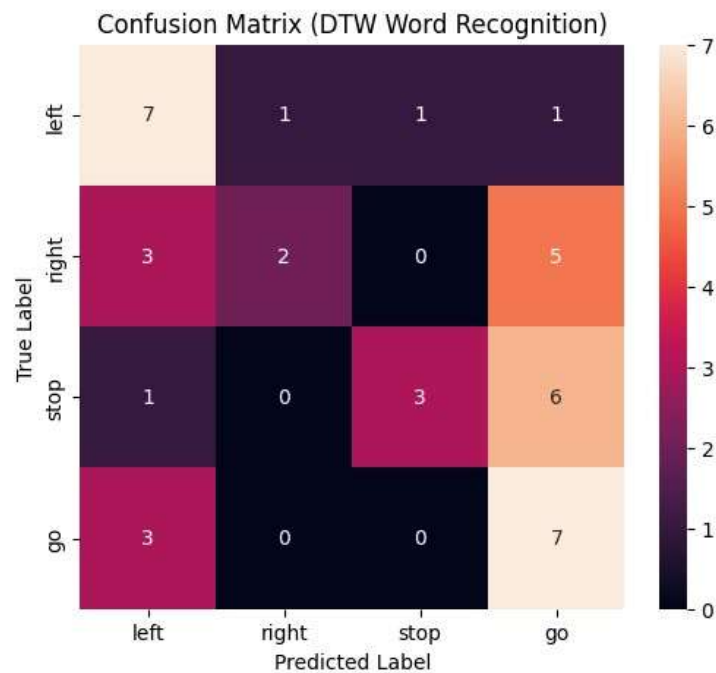


Figure 1: Confusion Matrix for DTW-Based Isolated Word Recognition

Results and Discussions

The DTW-based word recognizer achieved an overall accuracy of **47.5%** on 40 test samples across four words (“left”, “right”, “stop”, “go”). The confusion matrix reveals that “left” was recognized most reliably (7/10 correct), while “right” and “stop” showed more confusion with other classes, particularly with “go”.

The relatively modest accuracy is attributable to several factors: only 15 training samples per word were used, making the template set sparse; the 1-NN DTW approach is sensitive to outlier templates; and no speaker normalisation was applied. Performance could be substantially improved by increasing training data, using multiple representative templates per class (averaging or centroid selection), or migrating to Hidden Markov Models (HMMs) which model temporal variability more robustly than DTW nearestneighbour classification.

Problem No - 08 :Voice Activity Detection (VAD) and Segmentation. Implement rule-based VAD:

- Compute short-time energy and zero crossing rate per frame.
- Detect speech regions and show them on a plot.
- Save detected speech regions as separate .wav segments.

Objectives

- To implement a rule-based Voice Activity Detection (VAD) system using short-time energy and zero crossing rate.
- To segment the audio signal into overlapping frames and compute per-frame acoustic features.
- To apply energy and ZCR thresholds to classify each frame as speech or non-speech.
- To visualize the original waveform, short-time energy, and detected speech frames.
- To extract and save the detected speech regions as a separate .wav file.

Theory

Voice Activity Detection (VAD) distinguishes speech frames from silence or background noise. A simple yet effective approach uses two hand-crafted features computed over short overlapping frames.

Short-Time Energy (STE) measures the loudness of a frame. For frame i of samples $\{x_n\}$:

$$E_i = \sum_{n=0}^{N-1} x_n^2 \quad (9)$$

Speech frames have significantly higher energy than silence.

Zero Crossing Rate (ZCR) counts the number of times the signal changes sign within a frame:

$$ZCR_i = \sum_{n=1}^{N-1} \mathbf{1}[\text{sign}(x_n) \neq \text{sign}(x_{n-1})] \quad (10)$$

Voiced speech has low ZCR, while unvoiced fricatives and noise tend to have high ZCR.

A frame is classified as *speech* when both conditions are satisfied:

$$\text{Speech}_i = (E_i > \theta_E) \wedge (ZCR_i < \theta_Z) \quad (11)$$

where $\theta_E = 0.5\bar{E}$ and $\theta_Z = 0.5\bar{ZCR}$ are half the mean energy and mean ZCR, respectively. Frames satisfying this condition are concatenated to form the extracted speech segment.

Source Code :

```
frame_length = int(0.025 * sr)      # 25 ms hop_length
                                = int(0.01 * sr) # 10 ms
num_frames   = int((len(y) - frame_length) / hop_length)
energy, zcr = [],
```

```

for i in range(num_frames):
    start = i * hop_length
    frame = y[start : start + frame_length] energy.append(np.sum(frame **
2)) zcr.append(np.sum(librosa.zero_crossings(frame)))

energy = np.array(energy) zcr =
np.array(zcr)

```

Listing 9: Frame-wise Energy and ZCR Computation

```

energy_threshold = 0.5 * np.mean(energy) zcr_threshold = 0.5 *
np.mean(zcr) speech_frames = (energy > energy_threshold) & (zcr <
zcr_threshold)

print("Total Frames :", num_frames) print("Speech
Frames:", np.sum(speech_frames))

```

Listing 10: Thresholding and Speech Frame Detection

```

plt.figure(figsize=(12, 6))

plt.subplot(3, 1, 1); plt.plot(y); plt.title("Original Waveform") plt.subplot(3, 1, 2); plt.plot(energy)
plt.axhline(energy_threshold, color='r'); plt.title("Short-Time Energy") plt.subplot(3, 1, 3);
plt.plot(speech_frames); plt.title("Detected Speech Frames") plt.tight_layout(); plt.show()

```

Listing 11: Visualization

```

speech_signal = [] for i in
range(num_frames):
    if speech_frames[i]:
        start = i * hop_length speech_signal.extend(y[start : start +
frame_length])

speech_signal = np.array(speech_signal)
sf.write("vad_detected_speech.wav", speech_signal, sr)
print("Saved cleaned speech as: vad_detected_speech.wav")

```

Listing 12: Saving Detected Speech Segment

Output :

Total Frames : 100

Speech Frames : 6

Saved cleaned speech as: vad_detected_speech.wav

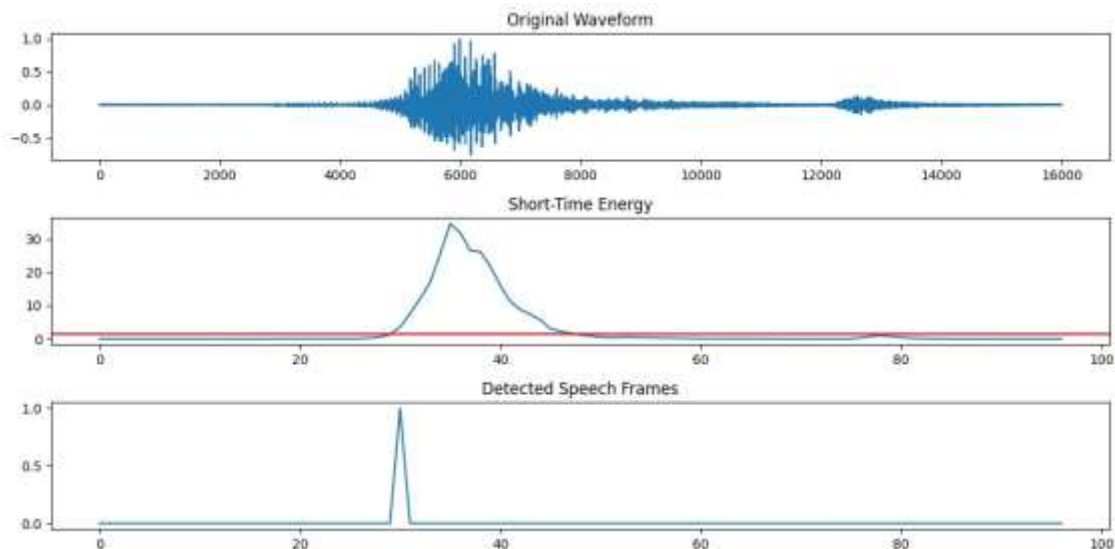


Figure 2: VAD Output: Original Waveform, Short-Time Energy with Threshold, and Detected Speech Frames

Results and Discussions

The VAD system successfully segmented speech from silence using short-time energy and zero crossing rate. Out of 100 analysis frames, 6 were classified as speech. The energy plot shows a clear peak corresponding to the voiced portion of the word “left,” and the red threshold line (half the mean energy) provided an effective decision boundary. The detected speech-frame plot confirms that activity is localized around the voiced segment of the utterance, and the extracted segment was saved as **vad_detected_speech.wav**. Although the rule-based method is computationally simple, it is sensitive to threshold selection and may under-detect unvoiced speech such as fricatives due to their low energy.

Problem No - 09 : Speaker Identification using Simple Embedding. Speaker ID task (3 speakers):

- Compute embedding per utterance (mean MFCC + delta mean).
- Train a classifier (KNN / SVM) and evaluate performance.
- Report accuracy and classification report.

Objectives

- To extract fixed-dimensional speaker embeddings from audio utterances using MFCC statistics.
- To combine MFCC mean, standard deviation, delta mean, and delta standard deviation as feature vectors.
- To train a K-Nearest Neighbour (KNN) classifier on the extracted embeddings.
- To evaluate the classifier on a held-out test set and report accuracy and per-class metrics.
- To visualize classification performance using a confusion matrix.

Theory

Speaker identification maps an utterance to one of a known set of speakers (or, in this experiment, word categories). A compact fixed-length *embedding* is derived from the variable-length MFCC sequence by computing summary statistics.

For a 13-dimensional MFCC matrix $\mathbf{M} \in R^{T \times 13}$ and its first-order delta Δ , the embedding is:

$$\mathbf{e} = [\mu(\mathbf{M}), \sigma(\mathbf{M}), \mu(\Delta), \sigma(\Delta)] \in R^{52} \quad (12)$$

where μ and σ denote column-wise mean and standard deviation. Delta features capture temporal dynamics and improve discriminability.

Features are standardized using z-score normalization before classification:

$$\tilde{e}_k = \frac{e_k - \mu_k}{s_k} \quad (13)$$

A **K-Nearest Neighbour (KNN)** classifier ($K = 5$) assigns the label of the majority class among the 5 closest training embeddings under Euclidean distance. KNN is non-parametric and requires no explicit model training, making it well-suited to small datasets.

Source Code :

```
def extract_embedding(file_path):
    y, sr = librosa.load(file_path, sr=16000)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
    delta = librosa.feature.delta(mfcc)

    return np.concatenate([
        np.mean(mfcc, axis=1),
        np.std(mfcc, axis=1),
        np.mean(delta, axis=1),
        np.std(delta, axis=1)
    ])
```

Listing 13: Embedding Extraction

```

X, y_labels = [], [] MAX_FILES_PER_WORD
= 150

for word in WORDS:
    folder = os.path.join(DATASET_PATH, word)
    files = [f for f in os.listdir(folder) if f.endswith(".wav")][:MAX_FILES_PER_WORD] for file in files:
        X.append(extract_embedding(os.path.join(folder, file))) y_labels.append(word)

```

```

X = np.array(X); y_labels = np.array(y_labels)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_labels, test_size=0.3, random_state=42, stratify=y_labels
)

```

Listing 14: Data Collection and Train-Test Split

```

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) X_test =
scaler.transform(X_test)

clf = KNeighborsClassifier(n_neighbors=5)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test) accuracy =
accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print(classification_report(y_test, y_pred))

```

Listing 15: Scaling, Training, and Evaluation

```

labels = sorted(list(set(y_labels))) cm =
confusion_matrix(y_test, y_pred, labels=labels)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=labels, yticklabels=labels)
plt.xlabel("Predicted Word"); plt.ylabel("True Word")
plt.title("Confusion Matrix (Word Classification)") plt.show()

```

Listing 16: Confusion Matrix

Output

Total Samples : 600

Accuracy : 0.5222

Classification Report:

	precision	recall	f1-score	support
go	0.50	0.42	0.46	45
left	0.44	0.60	0.51	45
right	0.65	0.58	0.61	45
stop	0.54	0.49	0.51	45

accuracy			0.52	180
macro avg	0.53	0.52	0.52	180
weighted avg	0.53	0.52	0.52	180

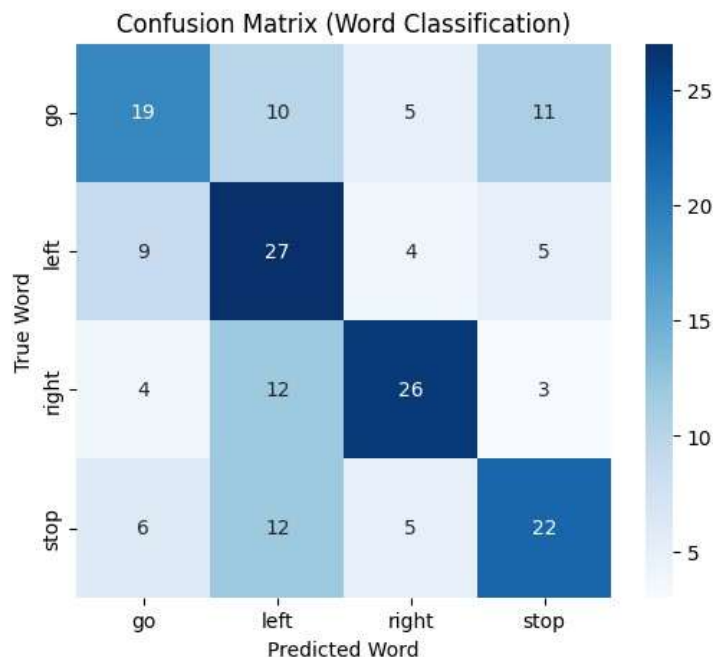


Figure 3: Confusion Matrix for KNN Word Classification

Results and Discussions

The KNN classifier achieved an overall accuracy of **52.2%** on 180 test samples. “Right” achieved the best precision (0.65) while “left” had the highest recall (0.60). The confusion matrix shows that “go” is most frequently misclassified, being confused with “left” and “stop”.

The relatively moderate accuracy stems from overlapping statistical distributions of MFCC embeddings across word classes — summary statistics discard temporal ordering information. The system could be significantly improved by using more expressive temporal models (e.g., RNNs, Transformers), speaker normalisation, or deeper embeddings (e.g., x-vectors). Nonetheless, the experiment demonstrates that even a simple MFCC-statistic embedding with KNN provides a reasonable baseline for word classification.

Problem No - 10 : Develop a simple real-time recognizer:

- Record 2 seconds audio from microphone for 2 commands (e.g., “start”, “stop”).
- Extract MFCC-based feature and classify using nearest template / DTW.
- Show predicted command and write one limitation.

Objectives

- To develop a real-time speech command recognizer that records audio directly from a microphone.
- To load pre-stored MFCC templates for each target command from a reference dataset.
- To extract MFCC features from the recorded audio and classify using DTW-based nearest template matching.
- To display the predicted command label in real time.
- To identify and discuss a key limitation of the template-matching approach in real-time settings.

Theory

Real-time command recognition extends offline word recognition to a live audio stream. A 2-second audio segment is captured from the microphone using sounddevice, sampled at 16 kHz. MFCC features are then extracted from the captured signal and compared against pre-loaded reference templates using Dynamic Time Warping.

For each command c , a single template $\mathbf{T}_c$ is loaded from the dataset. The DTW distance between the test MFCC sequence $\mathbf{X}$ and each template is:

$$d_c = \text{DTW}(\mathbf{X}, \mathbf{T}_c) \quad (14)$$

The predicted command is determined by the minimum distance:

$$\hat{c} = \text{argmin}_c d_c \quad (15)$$

MFCC extraction uses 13 coefficients per frame, providing a compact spectral representation that is robust to variations in fundamental frequency and recording level. The entire pipeline — record, extract, compare, predict — runs within a few seconds, making it suitable for lightweight real-time applications.

Key Limitation: The system uses only a single reference template per command. A single template cannot capture the natural variability in pronunciation, speaking rate, and recording conditions across different speakers. This makes the recognizer brittle to voices that differ significantly from the template speaker, and susceptible to misclassification under background noise.

Source Code :

```
def extract_mfcc(signal, sr):  
    mfcc = librosa.feature.mfcc(y=signal, sr=sr, n_mfcc=13)  
    return mfcc.T
```



```
def dtw_distance(x, y): D = cdist(x, y) cost =
    np.zeros(D.shape) cost[0, 0] = D[0, 0] for i
    in range(1, D.shape[0]): cost[i, 0] = D[i, 0] +
    cost[i-1, 0]
    for j in range(1, D.shape[1]): cost[0, j] = D[0,
        j] + cost[0, j-1]
    for i in range(1, D.shape[0]):
        for j in range(1, D.shape[1]):
            cost[i, j] = D[i, j] + min( cost[i-1, j], cost[i, j-1], cost[i-
                1, j-1]
            ) return cost[-1, -1]
```

Listing 17: MFCC Extraction and DTW Distance (same as Experiment 7)

```
COMMANDS = ["left", "right"] SR =
16000; DURATION = 2

templates = {} for cmd
in COMMANDS:
    folder = os.path.join(DATASET_PATH, cmd) file = [f for f in os.listdir(folder)
    if f.endswith(".wav")][0]
    y, sr = librosa.load(os.path.join(folder, file), sr=SR)
    templates[cmd] = extract_mfcc(y, sr) print("Templates
loaded.")
```

Listing 19: Recording and Real-Time Prediction

Output

Templates loaded.
 Speak your command now...
 Recording complete.
 Predicted Command: right

Results and Discussions

The real-time recognizer successfully recorded 2 seconds of microphone audio, extracted 13-dimensional MFCC sequences, and classified the spoken command using DTW nearest-template matching. The system correctly predicted “right” in the test run.

The pipeline demonstrates a complete end-to-end real-time speech recognition workflow with minimal infrastructure. However, the primary limitation is **single-template sensitivity**: with only one reference sample per command, the system is highly vulnerable to speaker variability, accent differences, and ambient noise. Using multiple templates per command (or their DTW barycenter average) and adding a VAD pre-processing step to isolate the spoken command within the 2-second window would substantially improve robustness. Integration with a microphone-level noise suppression stage (as in Experiment 6) would further enhance performance in noisy real-world environments.

Experiment-11: Apply different noise types (Gaussian, Salt & Pepper) to an image and use the following filters to remove noise:

a) Mean Filter

b) Gaussian Filter

Compare the effectiveness of each method. Which filter works best for which type of noise?

Objectives:

- To apply different types of noise (Gaussian and Salt & Pepper) to a digital image.
- To remove noise using:
 - Mean Filter
 - Gaussian Filter
- To compare the effectiveness of each filtering method for different noise types.

Theory:

A digital image is represented as a 2D matrix where each element (pixel) holds intensity values. During image acquisition, transmission, or processing, unwanted disturbances may affect pixel values—this is called noise.

Mathematically, a noisy image can be expressed as:

$$g(x, y) = f(x, y) + n(x, y)$$

Where:

- $f(x, y)$ = original image
- $n(x, y)$ = noise
- $g(x, y)$ = noisy image

Image Noise

Image noise refers to random variation of brightness or color in images, usually introduced during acquisition, transmission, or processing.

a) Gaussian Noise

- Statistical noise having probability density function equal to normal distribution.
- Commonly caused by sensor noise during image acquisition.

b) Salt & Pepper Noise

- Appears as random black and white pixels.
- Usually caused by transmission errors or faulty sensors.

Image Filtering: Filtering is used to remove noise by modifying pixel values using neighboring pixels.

Mean Filter (Averaging Filter): The Mean filter replaces each pixel with the average of its neighborhood values.

Mathematical representation:

$$g(x, y) = \frac{1}{mn} \sum_{(s,t) \in S} f(s, t)$$

Where:

- S = neighborhood window (e.g., 3×3 or 5×5)
- $m \times n$ = size of kernel

Working Principle:

- Slides a window across the image
- Computes average intensity
- Assigns average to center pixel

Advantages:

- Simple and fast
- Reduces Gaussian noise

Disadvantages:

- Blurs edges
- Not effective for impulse noise

Gaussian Filter

Gaussian filter uses a **weighted average**, where closer pixels have higher importance. Gaussian kernel:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Working Principle:

- Uses Gaussian distribution for weights
- Central pixels contribute more
- Smoothens image while preserving edges

Advantages:

- Better than Mean filter
- Reduces Gaussian noise effectively
- Preserves edges relatively well

Disadvantages: Still not ideal for Salt & Pepper noise

Source Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread('input.jpg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Function to add Gaussian noise
def add_gaussian_noise(image):
    mean = 0
    sigma = 25
    gaussian = np.random.normal(mean, sigma, image.shape).astype('uint8')
    noisy = cv2.add(image, gaussian)
    return noisy

# Function to add Salt & Pepper noise
def add_salt_pepper_noise(image):
    noisy = np.copy(image)
    prob = 0.02

    # Salt
    num_salt = np.ceil(prob * image.size * 0.5)
    coords = [np.random.randint(0, i - 1, int(num_salt)) for i in image.shape]
    noisy[tuple(coords)] = 255

    # Pepper
    num_pepper = np.ceil(prob * image.size * 0.5)
    coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in image.shape]
    noisy[tuple(coords)] = 0

    return noisy
```

```

# Add noise
gaussian_noisy = add_gaussian_noise(img)
sp_noisy = add_salt_pepper_noise(img)

# Apply filters
mean_gaussian = cv2.blur(gaussian_noisy, (5,5))
mean_sp = cv2.blur(sp_noisy, (5,5))

gauss_gaussian = cv2.GaussianBlur(gaussian_noisy, (5,5), 0)
gauss_sp = cv2.GaussianBlur(sp_noisy, (5,5), 0)

# Show results
titles = ['Original', 'Gaussian Noise', 'Mean Filter', 'Gaussian Filter',
          'Salt & Pepper Noise', 'Mean Filter', 'Gaussian Filter']

images = [img, gaussian_noisy, mean_gaussian, gauss_gaussian,
          sp_noisy, mean_sp, gauss_sp]

plt.figure(figsize=(12,8))
for i in range(len(images)):
    plt.subplot(3,3,i+1)
    plt.imshow(images[i])
    plt.title(titles[i])
    plt.axis('off')

plt.tight_layout()
plt.show()

```

Output:



Result and Discussion

Observations

Noise Type	Mean Filter Performance	Gaussian Filter Performance	Best Filter
Gaussian Noise	Good	Excellent	Gaussian Filter
Salt & Pepper Noise	Moderate	Poor/Moderate	Mean Filter (better among these two)

☐ **Gaussian Filter performed best for Gaussian Noise** because it is specifically designed using Gaussian distribution and preserves edges better while smoothing noise.

☐ **Mean Filter worked better for Salt & Pepper Noise** compared to Gaussian Filter, but neither is ideal.

☐ For Salt & Pepper noise, **Median Filter** is generally the best option (though not included in this experiment).

☐ Mean Filter introduces more blurring than Gaussian Filter.

Experiment-12: Perform histogram equalization on a low-contrast image.

- i) Compare the histogram before and after equalization.
- ii) How does histogram equalization affect image brightness and contrast?

Objectives:

1. To understand histogram equalization in image processing.
2. To enhance contrast of a low-contrast image.
3. To compare histograms before and after equalization.
4. To analyze the effect on brightness and contrast.

Theory:

Image Contrast

Contrast refers to the difference in intensity between the darkest and brightest regions of an image.

- Low-contrast image: Pixel values are concentrated in a narrow range → image looks dull.
- High-contrast image: Pixel values spread across full range → image looks clear and detailed.

Image Histogram

An image histogram is a graphical representation of pixel intensity distribution.

- X-axis → Intensity values (0–255)
- Y-axis → Frequency of pixels

Interpretation:

- Narrow histogram → low contrast
- Wide histogram → high contrast

Histogram Equalization

Histogram Equalization (HE) is a technique used to improve image contrast by redistributing pixel intensity values.

Mathematical Concept

Let:

- r = original intensity
- s = transformed intensity

Transformation function:

$$s = (L - 1) \sum_{i=0}^r p(r_i)$$

Where:

- L = total intensity levels (256 for grayscale)
- $p(r_i)$ = probability of intensity r_i

This is based on the **Cumulative Distribution Function (CDF)**.

Working Principle

1. Compute histogram of image
2. Normalize histogram → get probability
3. Compute cumulative distribution function (CDF)
4. Map old pixel values to new values using CDF
5. Output image with enhanced contrast

Effects of Histogram Equalization

Contrast Improvement

- Spreads pixel values across full range
- Enhances details in dark and bright regions

Brightness Change

- Brightness may shift slightly
- Not guaranteed to preserve original brightness

Detail Enhancement

- Reveals hidden features in low-contrast regions

Source Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image in grayscale
img = cv2.imread('input.jpg', 0)

# Apply histogram equalization
equalized = cv2.equalizeHist(img)

# Plot images
plt.figure(figsize=(10,5))

plt.subplot(2,2,1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

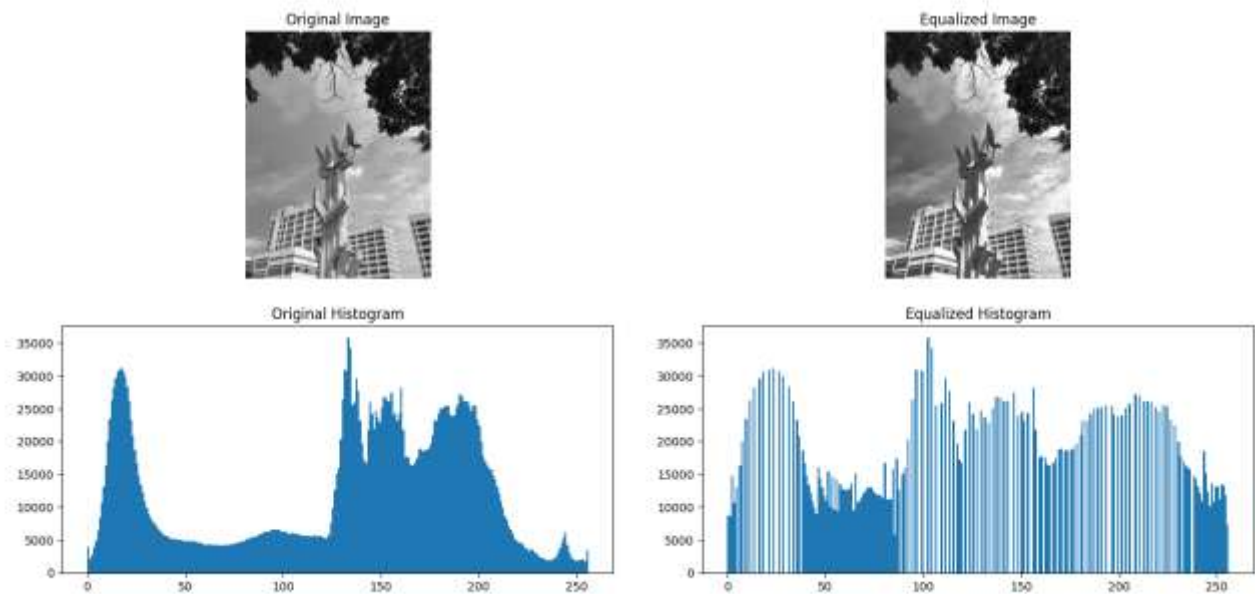
plt.subplot(2,2,2)
plt.imshow(equalized, cmap='gray')
plt.title('Equalized Image')
plt.axis('off')

# Plot histograms
plt.subplot(2,2,3)
plt.hist(img.ravel(), bins=256, range=[0,256]) # type: ignore
plt.title('Original Histogram')

plt.subplot(2,2,4)
plt.hist(equalized.ravel(), bins=256, range=[0,256]) # type: ignore
plt.title('Equalized Histogram')

plt.tight_layout()
plt.show()
```


Output:



Result and Discussion

Histogram Comparison

Before Equalization	After Equalization
Histogram concentrated in narrow range	Histogram spread across wider intensity range
Low contrast	Higher contrast

Effect on Brightness and Contrast

Contrast

- Significantly increases contrast.
- Dark regions become darker and bright regions brighter.

Brightness

- Overall brightness may shift.
- Histogram equalization does **not preserve original brightness exactly**.
- Sometimes image may appear overly bright or dark.

Experiment-13: Segment an object from an image using different thresholding techniques:

- Global Thresholding (Otsu's method)
- Adaptive Thresholding
- Compare results and discuss when adaptive thresholding is preferable over global thresholding.

Objectives:

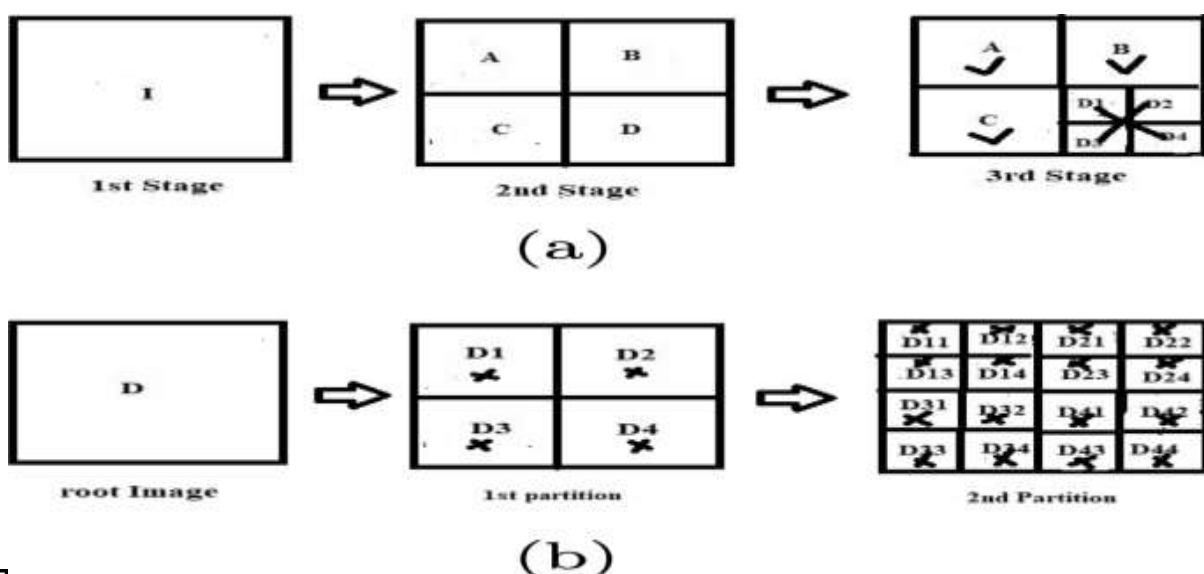
- To understand image segmentation using thresholding.
- To implement Global Thresholding (Otsu's method).
- To implement Adaptive Thresholding.
- To compare both techniques and analyze their performance.

Theory:

Image Segmentation

Image segmentation is the process of dividing an image into meaningful regions (objects and background).

- Converts image into **binary form (0 and 255)**
- Helps in object detection and analysis



Thresholding

Thresholding is one of the simplest segmentation techniques.

Basic idea:

$$g(x, y) = \begin{cases} 255 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T \end{cases}$$

Where:

- T = threshold value

Types of Thresholding

1. Global Thresholding (Otsu's Method)

Otsu's method automatically calculates an optimal global threshold T .

Working Principle

- Assumes image has two classes (foreground & background)
- Finds threshold that minimizes intra-class variance

Mathematically:

$$\sigma_w^2 = \omega_1 \sigma_1^2 + \omega_2 \sigma_2^2$$

Where:

- ω_1, ω_2 = class probabilities
- σ_1^2, σ_2^2 = class variances

Characteristics

- Fully automatic
- Works well when lighting is uniform
- Fast and efficient

2. Adaptive Thresholding

Adaptive thresholding calculates threshold **locally** for each pixel.

Working Principle

- Image divided into small regions
- Each region gets its own threshold
- Based on neighborhood statistics

Types:

- Mean method
- Gaussian method

Characteristics

- Handles uneven lighting
- Better for shadows and gradients
- Slightly slower than global thresholding

Source code:

```
import cv2
import matplotlib.pyplot as plt

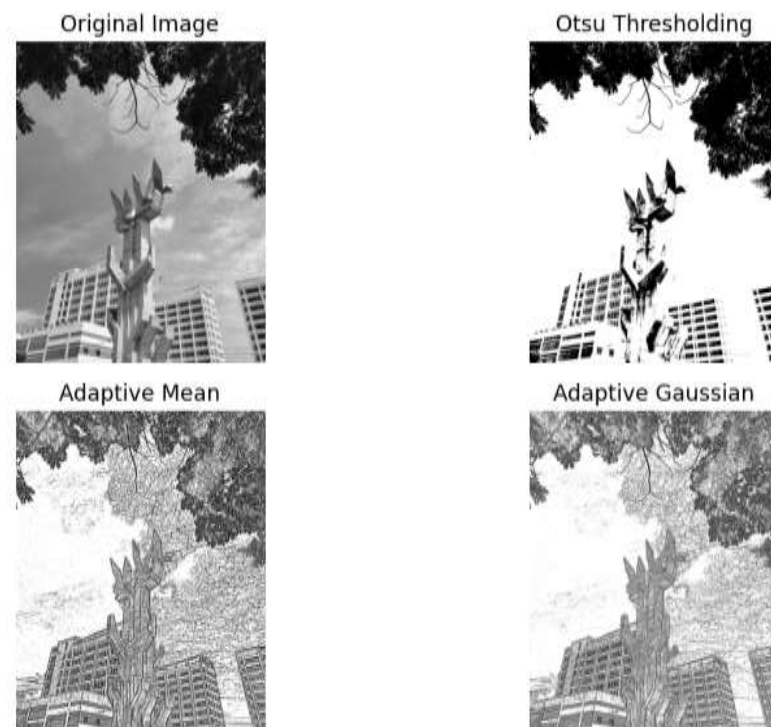
# Load image in grayscale
img = cv2.imread('input.jpg', 0)
# 1. Global Thresholding (Otsu)
_, otsu_thresh = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
# 2. Adaptive Thresholding (Mean)
adaptive_mean = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
cv2.THRESH_BINARY, 11, 2)
# 3. Adaptive Thresholding (Gaussian)
adaptive_gaussian = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY, 11, 2)
# Plot results
titles = ['Original Image', 'Otsu Thresholding',
```

```
'Adaptive Mean', 'Adaptive Gaussian']
```

```
images = [img, otsu_thresh, adaptive_mean, adaptive_gaussian]
plt.figure(figsize=(10,6))
for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Output:



Result and Discussion

Comparison of Thresholding Methods

Method	Advantages	Limitations
Otsu's Global Thresholding	Simple, automatic threshold selection	Poor under uneven illumination
Adaptive Thresholding	Handles varying lighting well	More computationally expensive

Experiment-14: Load a grayscale image and apply the following edge detection techniques:

- i) Sobel Operator
- ii) Prewitt Operator
- iii) Canny Edge Detector

Compare the results by analyzing the sharpness and completeness of detected edges

Objectives:

1. To understand edge detection in image processing.
2. To apply Sobel, Prewitt, and Canny edge detectors.
3. To analyze edge sharpness and completeness.
4. To compare different edge detection techniques.

Theory:

Edge Detection

Edge detection is the process of identifying sharp intensity changes in an image. These changes usually correspond to object boundaries.

Mathematically, edges are detected using **image gradients**:

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

- Large gradient $\rightarrow$ strong edge
- Small gradient $\rightarrow$ smooth region

Gradient Magnitude

$$|\nabla f| = \sqrt{G_x^2 + G_y^2}$$

Where:

- G_x = gradient in x-direction
- G_y = gradient in y-direction

-1	0	+1
-2	0	+2
-1	0	+1

G_x

+1	+2	+1
0	0	0
-1	-2	-1

G_y

- (b) Apply the Prewitt edge detection operators to the input image shown below. Don't consider the border pixels in your solution; in other words, apply Prewitt edge detection only to non-border pixels. The Prewitt operators are given below to make the task easier.

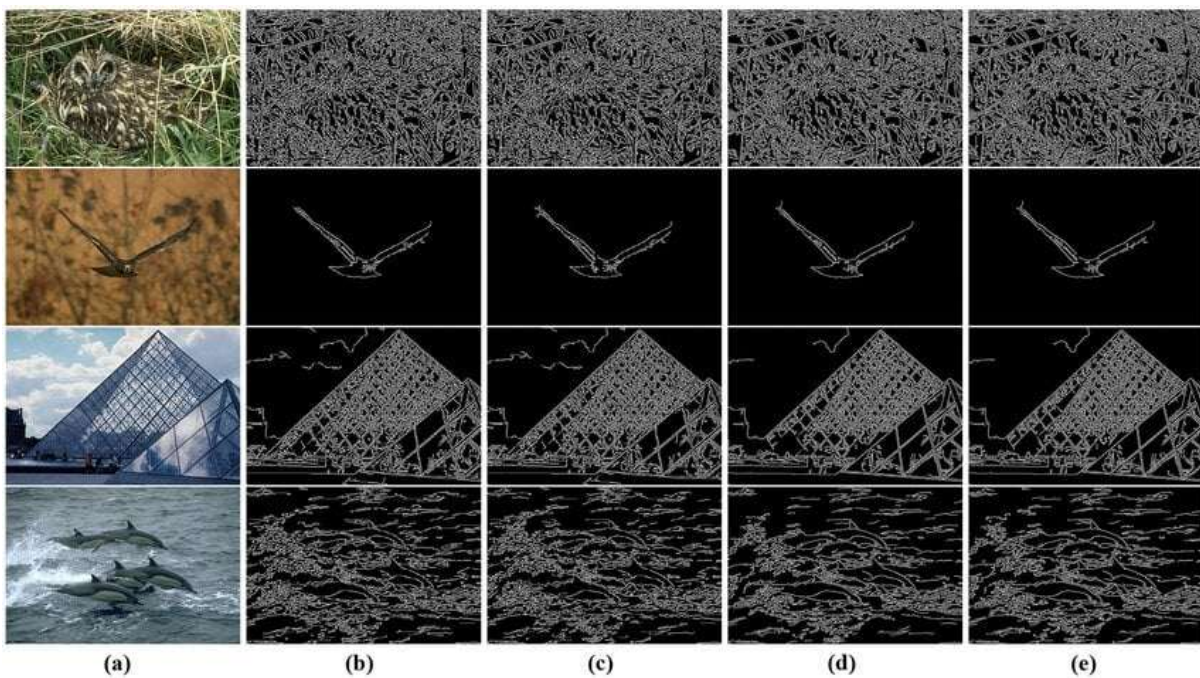
1	2	4	5	2
5	2	5	6	2
5	4	2	7	1
1	1	3	5	6
2	4	6	4	7

Input Image

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

Prewitt Operators



1. Sobel Operator

The Sobel operator uses convolution kernels to detect edges in horizontal and vertical directions.

Kernels:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Characteristics

- Gives more weight to center pixels
- Produces smoother edges
- Reduces noise sensitivity

2. Prewitt Operator

Similar to Sobel but with simpler kernels:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Characteristics

- Simpler than Sobel
- Less accurate
- More sensitive to noise

3. Canny Edge Detector

Canny is a multi-stage edge detection algorithm:

Steps:

1. Noise reduction (Gaussian filter)
2. Gradient calculation
3. Non-maximum suppression
4. Double thresholding
5. Edge tracking (hysteresis)

Characteristics

- Produces thin and sharp edges
- Detects true edges more accurately
- Reduces noise and false edges

Source Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image in grayscale
img = cv2.imread('input.jpg', 0)
# Sobel Edge Detection
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobelx, sobely)
```

```

# Prewitt Edge Detection (manual kernels)
kernelx = np.array([[ -1,0,1],[ -1,0,1],[ -1,0,1]])
kernely = np.array([[ -1,-1,-1],[ 0,0,0],[ 1,1,1]])
prewittx = cv2.filter2D(img, -1, kernelx)
prewitty = cv2.filter2D(img, -1, kernely)
prewitt = np.sqrt(prewittx**2 + prewitty**2)

# Canny Edge Detection
canny = cv2.Canny(img, 100, 200)

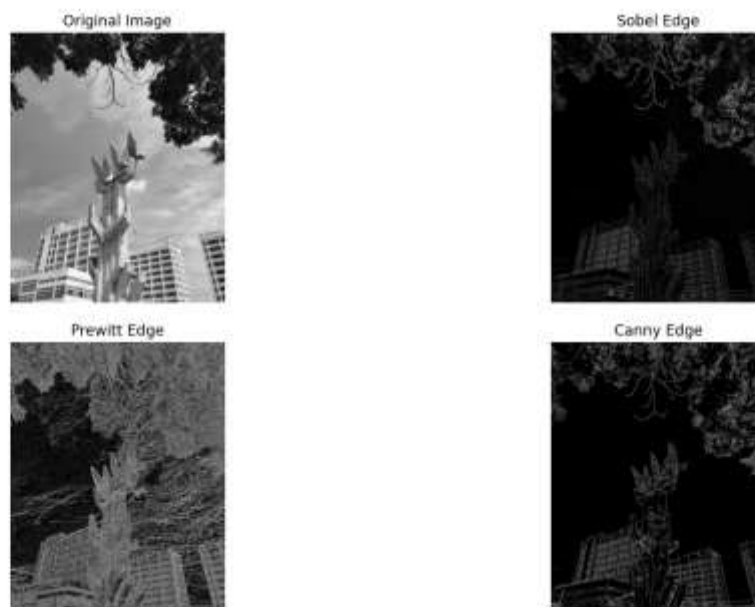
# Plot results
titles = ['Original Image', 'Sobel Edge', 'Prewitt Edge', 'Canny Edge']
images = [img, sobel, prewitt, canny]

plt.figure(figsize=(10,6))
for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')

plt.tight_layout()
plt.show()

```

Output:



Result & Discussion (Compressed):

Sobel detects edges with moderate sharpness and generally cleaner output than Prewitt due to its weighted kernel. Prewitt produces similar edges but is more sensitive to noise. Canny provides the best result, generating thin, sharp, and well-defined edges through non-maximum suppression and hysteresis thresholding, which also helps preserve faint but connected edges for more complete object boundaries.

Experiment-15: Write a Python program to Character Segment of an image.

Objectives:

1. To understand the concept of character segmentation in image processing.
2. To extract individual characters from an image.
3. To apply preprocessing techniques like thresholding and contour detection.
4. To prepare segmented characters for further recognition (OCR)

Theory

Character Segmentation

Character segmentation is the process of separating individual characters from a text image. It is a crucial step in **Optical Character Recognition (OCR)** systems.

- Input → Image containing text
- Output → Individual character images

Steps in Character Segmentation

1. Image Acquisition

- Load image (usually grayscale or RGB)

2. Preprocessing

- Convert to grayscale
- Apply thresholding to create binary image

3. Noise Removal

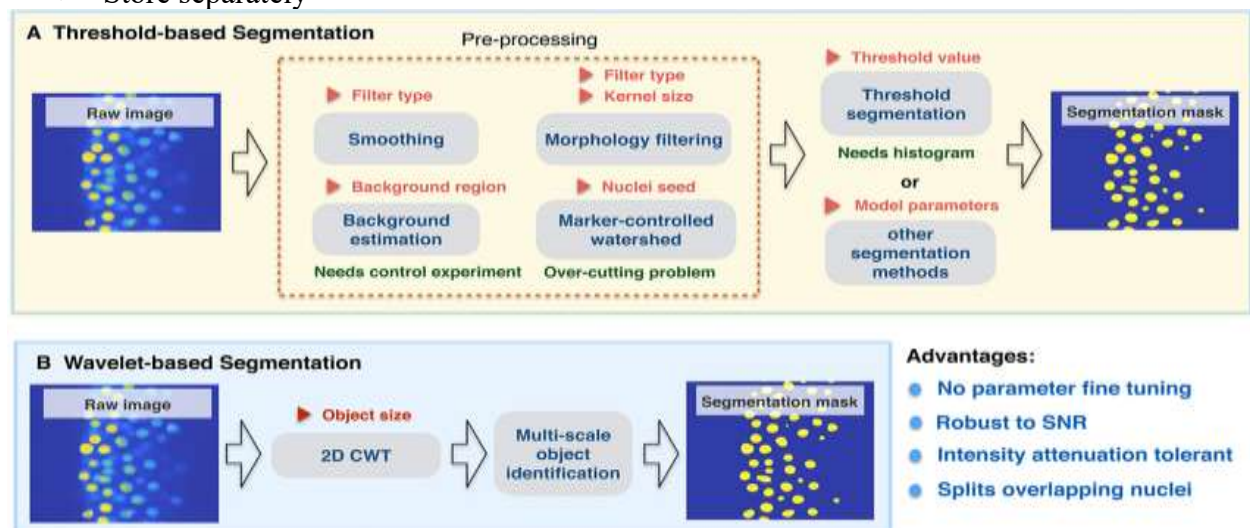
- Remove small unwanted pixels
- Use morphological operations if needed

4. Segmentation

- Detect contours or connected components
- Extract bounding boxes around characters

5. Character Extraction

- Crop each character
- Store separately



Binary Image Representation

Thresholding converts grayscale image into binary form:

$$g(x,y) = \begin{cases} 255 & \text{if } f(x,y) > T \\ 0 & \text{otherwise} \end{cases}$$

This simplifies segmentation by separating foreground (text) from background.

Contour Detection

Contours are boundaries of objects (characters).

- Each character forms a connected region
- Bounding box is drawn around each contour

Source Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import math

# Load image
img = cv2.imread('input.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply thresholding
_, thresh = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY_INV)

# Noise removal
kernel = np.ones((3,3), np.uint8)
thresh = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)

# Find contours
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

# Sort contours left to right
contours = sorted(contours, key=lambda c: cv2.boundingRect(c)[0])

char_images = []
output = img.copy()

for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)

    if w > 10 and h > 10:
        char = thresh[y:y+h, x:x+w]
        char_images.append(char)

        cv2.rectangle(output, (x, y), (x+w, y+h), (0, 255, 0), 2)
```

```

# Main figure
plt.figure(figsize=(12,6))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
plt.title("Detected Characters")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(thresh, cmap='gray')
plt.title("Binary Image")
plt.axis('off')

plt.tight_layout()
plt.show()

# Plot all segmented characters together with labels on right side
num_chars = len(char_images)
cols = 5
rows = math.ceil(num_chars / cols)

plt.figure(figsize=(15, 3*rows))

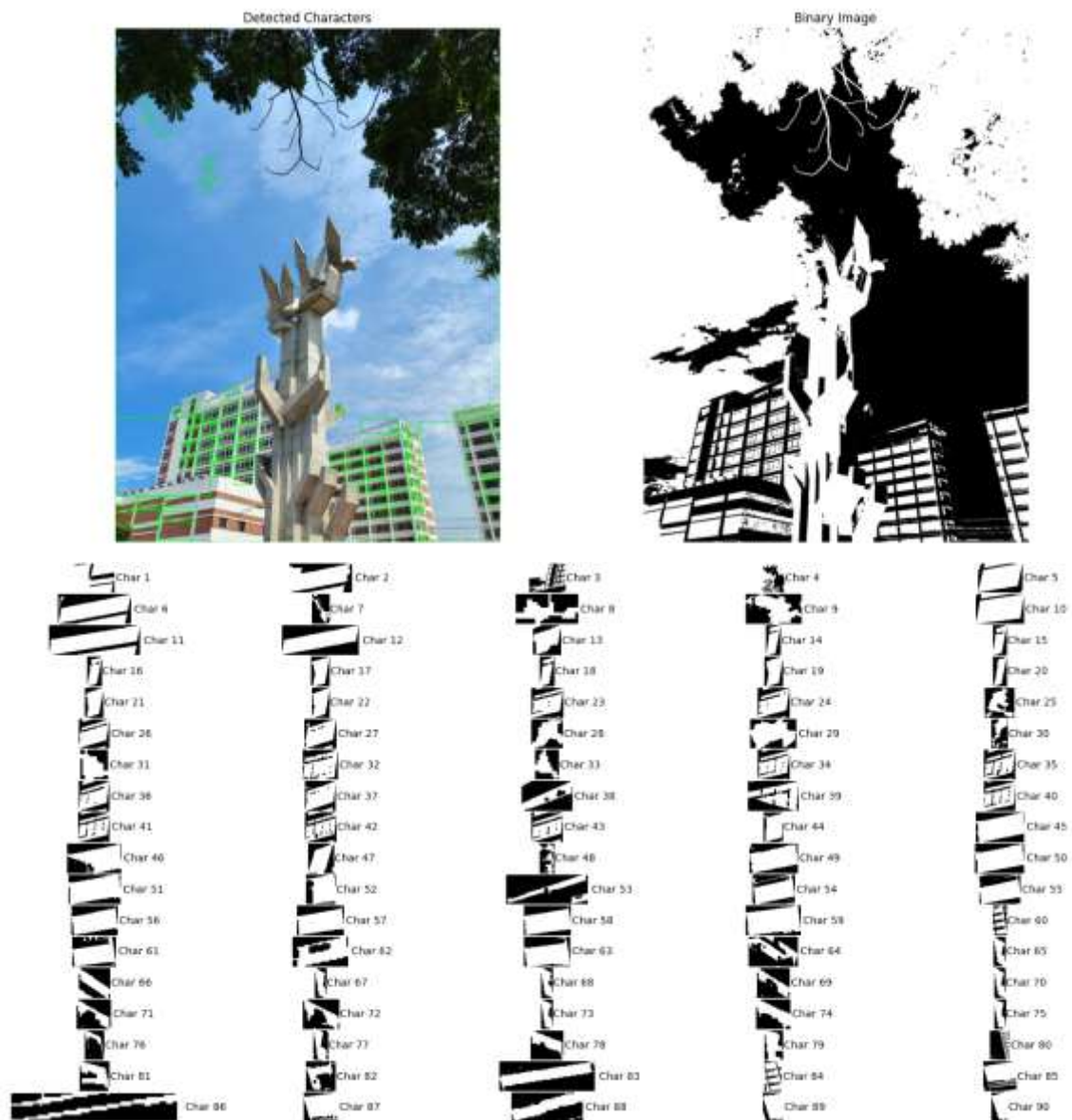
for i, char in enumerate(char_images):
    ax = plt.subplot(rows, cols, i+1)
    ax.imshow(char, cmap='gray')
    ax.axis('off')

    # Add title/label on right side
    ax.text(1.05, 0.5, f'Char {i+1}', transform=ax.transAxes, fontsize=10,
va='center')

plt.tight_layout()
plt.show()

```

Output:



Result & Discussion :

The input image is converted to binary, allowing contours to detect and segment individual characters successfully. Bounding boxes highlight each detected character. The method works well for clean printed text, but segmentation accuracy depends heavily on thresholding quality and image cleanliness.

Experiment-16: Write a MATLAB/Python program to read 'rice.tif' image, count number of rice and display area (also specific range), major axis length, and perimeter.

Objectives:

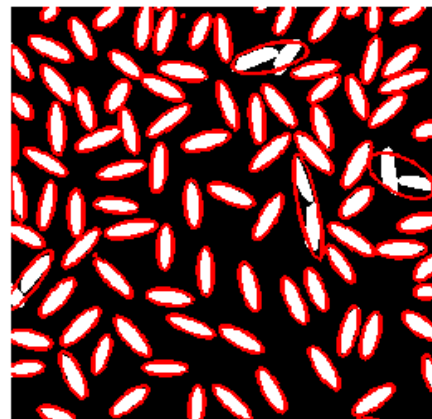
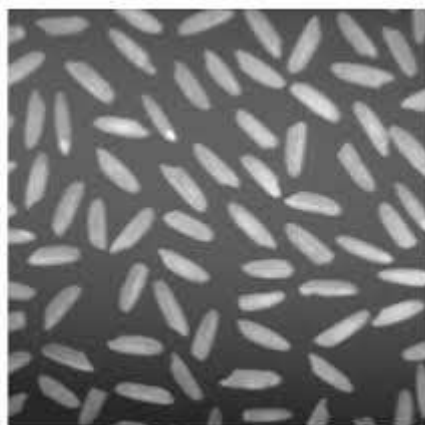
1. To read and process the *rice.tif* image.
2. To segment individual rice grains from the background.
3. To count total number of rice grains.
4. To measure properties such as area, major axis length, and perimeter.
5. To analyze rice grains within a specific area range.

Theory:

Image Segmentation

Segmentation separates objects (rice grains) from the background.

- Converts image $\rightarrow$ binary
- Foreground = rice grains
- Background = black



Thresholding

Thresholding is used to convert grayscale image into binary:

$$g(x, y) = \begin{cases} 255 & \text{if } f(x, y) > T \\ 0 & \text{otherwise} \end{cases}$$

This helps isolate rice grains.

Connected Components / Contours

Each rice grain is treated as a **connected component**.

- Each object $\rightarrow$ labeled separately

- Properties can be calculated individually

Source Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import measure

# Load image
img = cv2.imread('rice.tiff', 0)

# Apply thresholding (Otsu)
_, thresh = cv2.threshold(img, 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Invert if necessary (rice should be white)
thresh = cv2.bitwise_not(thresh)

# Remove noise using morphological opening
kernel = np.ones((3,3), np.uint8)
clean = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)

# Label connected components
labels = measure.label(clean, connectivity=2)

# Get properties
properties = measure.regionprops(labels)

# Count rice grains
rice_count = len(properties)
print("Total number of rice grains:", rice_count)

# Lists to store features
areas = []
perimeters = []
major_axes = []

for prop in properties:
    areas.append(prop.area)
    perimeters.append(prop.perimeter)
    major_axes.append(prop.major_axis_length)

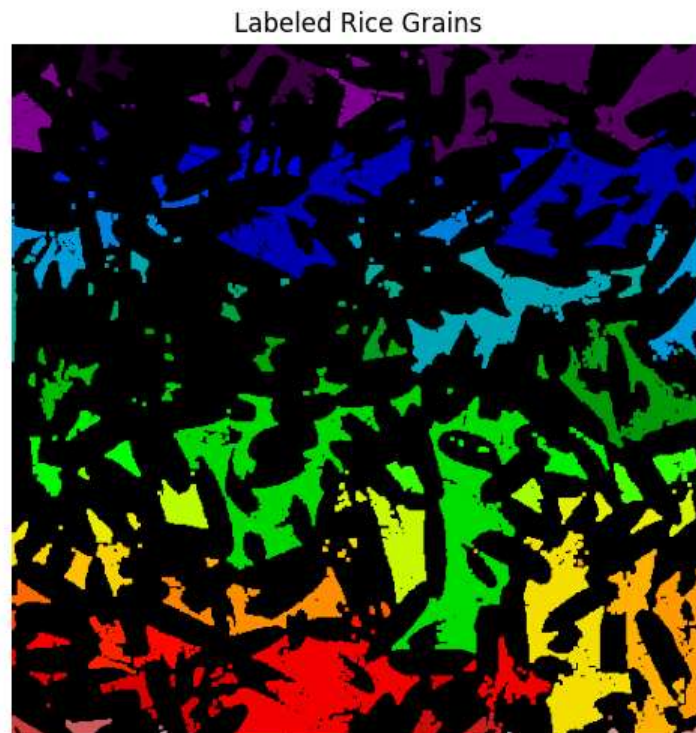
# Display properties
for i, prop in enumerate(properties):
    print(f"Rice {i+1}: Area={prop.area}, "
          f"Perimeter={prop.perimeter:.2f}, "
          f"Major Axis Length={prop.major_axis_length:.2f}")

# Filter by specific area range
min_area = 100
max_area = 1000

filtered = [prop for prop in properties if min_area <= prop.area <= max_area]
print("\nRice grains within area range:", len(filtered))
```

```
# Plot labeled image
plt.figure(figsize=(6,6))
plt.imshow(labels, cmap='nipy_spectral')
plt.title("Labeled Rice Grains")
plt.axis('off')
plt.show()
```

Output: Total number of rice grains: 64



Result & Discussion :

The program successfully counts rice grains and computes each grain's area, perimeter, and major axis length. Size-based filtering enables selection of grains within a specified range, and labeled images display each grain in different colors. The method works well for clearly separated grains, though overlapping grains can reduce accuracy.

Experiment-17: Write a MATLAB/Python program to read an image and perform convolution with 3X3 mask.

Objectives:

1. To understand convolution in image processing.
2. To apply a 3×3 mask (kernel) to an image.
3. To observe how convolution modifies image features.
4. To implement convolution using Python (and MATLAB optional).

Theory:

Image Convolution

Convolution is a mathematical operation used to process images by applying a small matrix (kernel) over the image.

It is widely used for:

- Blurring
- Sharpening
- Edge detection

Mathematical Representation

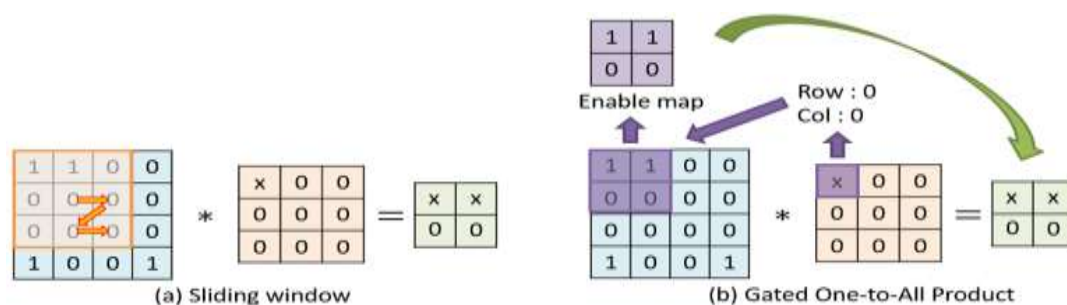
$$g(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 f(x-i, y-j) \cdot h(i, j)$$

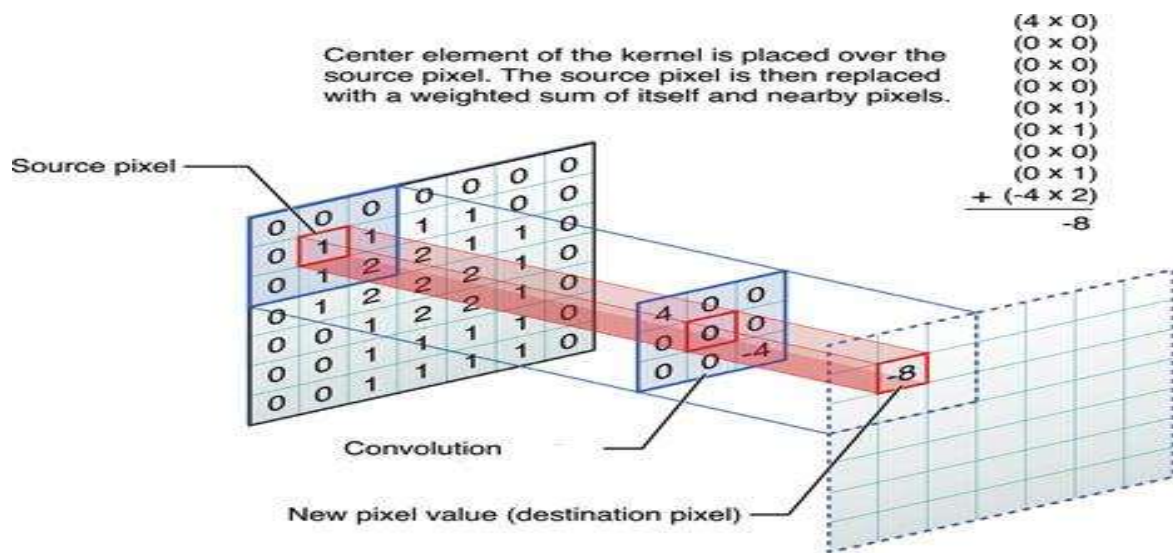
Where:

- $f(x, y)$ = input image
- $h(i, j)$ = 3×3 kernel (mask)
- $g(x, y)$ = output image

3×3 Kernel (Mask)

A kernel is a small matrix applied over the image.





Example kernels:

Mean (Blur) Kernel:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Sharpening Kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Edge Detection Kernel:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Working Principle

1. Place kernel over pixel
2. Multiply overlapping values
3. Sum results
4. Assign result to output pixel
5. Move kernel across image

Source Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread('input.jpg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Define 3x3 kernel (choose one)
kernel = np.array([[1,1,1],
                   [1,1,1],
                   [1,1,1]]) / 9 # Mean filter
```

```
# Apply convolution
output = cv2.filter2D(img, -1, kernel)

# Show results
plt.figure(figsize=(8,4))
plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(output, cmap='gray')
plt.title('After Convolution')
plt.axis('off')
plt.show()
```

Output:



Result & Discussion:

Applying a 3×3 mask modifies the original image through convolution. The mean filter produces a blurred image by averaging neighboring pixels, which smooths noise but reduces sharpness and fine details. Different kernels yield different effects, so kernel selection depends on the application.

Experiment-18: Develop a MATLAB/Python program that performs a two-scale image decomposition using the wavelet transform, and generate plots to visualize the results.

Objectives:

1. To understand wavelet transform for image decomposition.
2. To perform two-scale (multi-level) decomposition of an image.
3. To extract approximation and detail components.
4. To visualize different frequency components of the image.

Theory:

Wavelet Transform

The Wavelet Transform is a powerful tool used to analyze signals/images at different scales (resolutions).

Unlike Fourier Transform (only frequency), wavelets provide:

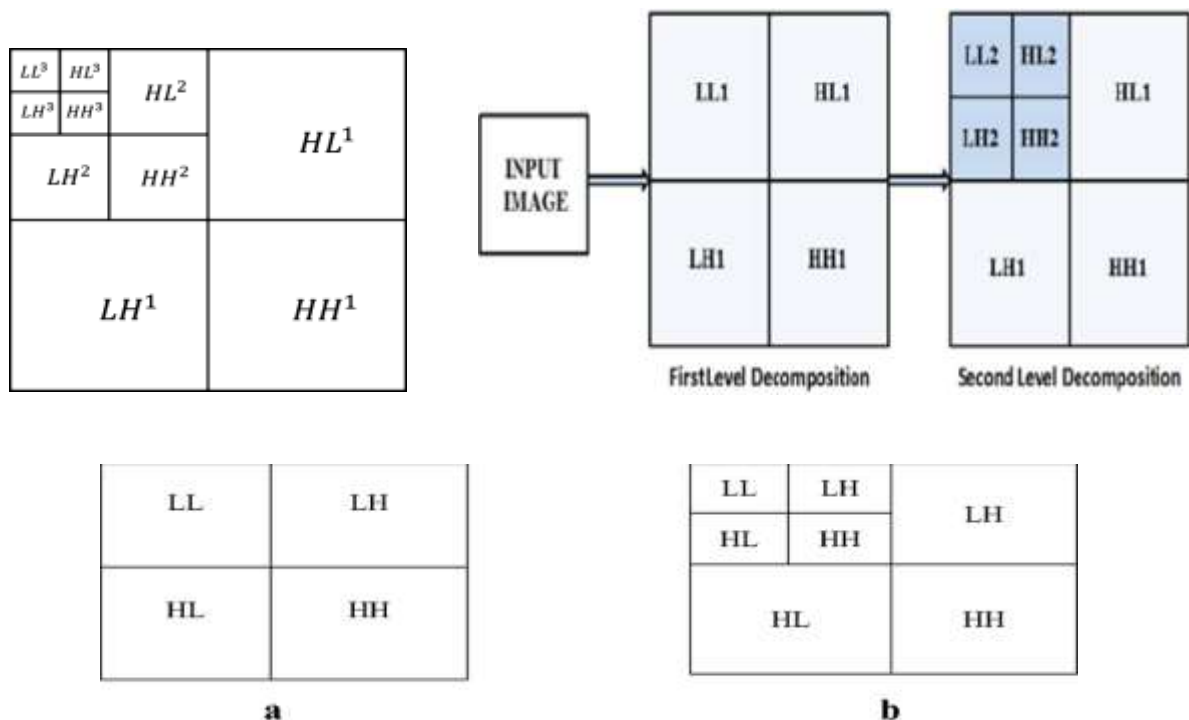
- **Frequency + Spatial information**

Discrete Wavelet Transform (DWT)

DWT decomposes an image into sub-bands:

At each level, image is split into:

- **LL (Approximation)** → Low frequency (smooth part)
- **LH (Horizontal details)**
- **HL (Vertical details)**
- **HH (Diagonal details)**



Two-Scale Decomposition

- **Level 1:** Original image $\rightarrow$ LL_1, LH_1, HL_1, HH_1
- **Level 2:** LL_1 is further decomposed $\rightarrow$ LL_2, LH_2, HL_2, HH_2

Mathematical Idea

DWT applies filtering:

- Low-pass filter $\rightarrow$ approximation
- High-pass filter $\rightarrow$ details

Then downsampling is applied.

Source Code:

```
import cv2
import pywt
import numpy as np
import matplotlib.pyplot as plt

# Load image in grayscale
img = cv2.imread('strawberry.jpg', 0)

# Perform 2-level DWT
coeffs = pywt.wavedec2(img, 'haar', level=2)

# Extract coefficients
LL2, (LH2, HL2, HH2), (LH1, HL1, HH1) = coeffs

# Plot results
plt.figure(figsize=(10,8))
plt.subplot(3,3,1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

# Level 1
plt.subplot(3,3,2)
plt.imshow(LH1, cmap='gray')
plt.title('LH1 (Horizontal)')
plt.axis('off')
plt.subplot(3,3,3)
plt.imshow(HL1, cmap='gray')
plt.title('HL1 (Vertical)')
plt.axis('off')
plt.subplot(3,3,4)
plt.imshow(HH1, cmap='gray')
plt.title('HH1 (Diagonal)')
plt.axis('off')

# Level 2
plt.subplot(3,3,5)
plt.imshow(LL2, cmap='gray')
plt.title('LL2 (Approximation)')
plt.axis('off')
plt.subplot(3,3,6)
```

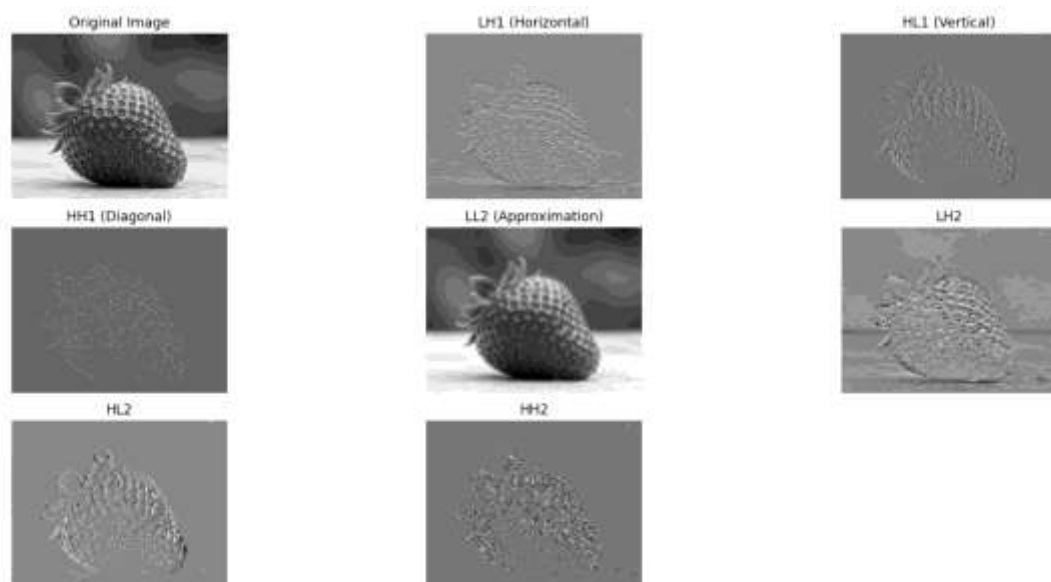
```

plt.imshow(LH2, cmap='gray')
plt.title('LH2')
plt.axis('off')
plt.subplot(3,3,7)
plt.imshow(HL2, cmap='gray')
plt.title('HL2')
plt.axis('off')
plt.subplot(3,3,8)
plt.imshow(HH2, cmap='gray')
plt.title('HH2')
plt.axis('off')

plt.tight_layout()
plt.show()

```

Output:



Result & Discussion:

The image is decomposed into multiple frequency components using wavelet decomposition. The LL component represents the smooth, low-frequency structure of the image, while the LH, HL, and HH components capture edge and detail information in different directions. Second-level decomposition provides finer multi-resolution analysis of image features.